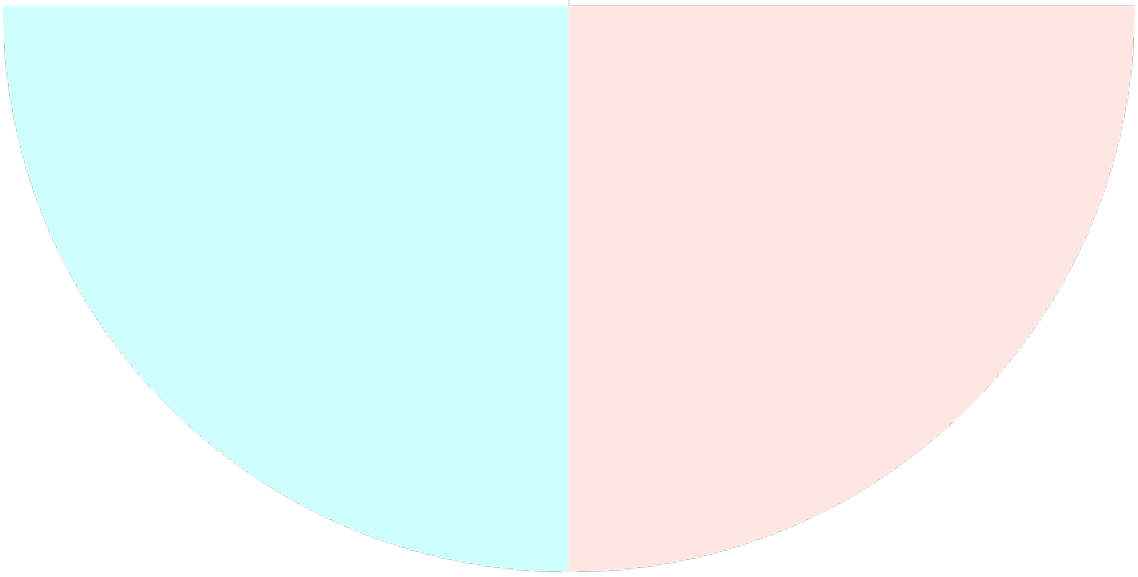
The Globe oposiciones

Bloque 3 - Tema 7

ARQUITECTURA DE SISTEMAS CLIENTE/SERVIDOR Y MULTICAPAS: COMPONENTES Y OPERACIÓN. ARQUITECTURAS DE SERVICIOS WEB Y PROTOCOLOS ASOCIADOS



PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE 2

1. ARQUITECTURAS DE SISTEMAS C/S Y MULTICAPA 3
   1. *Componentes 3*
   2. *Arquitectura mononivel o de 1 nivel 5*
   3. *Arquitectura cliente/servidor o de 2 niveles 5*
   4. *Arquitectura de 3 niveles 7*
   5. *Arquitectura multicapa y arquitecturas de 3 niveles y multinivel 9*
2. ARQUITECTURA ORIENTADA A SERVICIOS (SOA) 12
3. ARQUITECTURA DE MICROSERVICIOS 15
4. ARQUITECTURA DE SERVICIOS WEB 21
   1. *Tipos de servicios web 21*
5. SERVICIOS WEB SOAP 23
   1. *Arquitectura 23*
   2. *Tecnologías 24*
   3. *SOAP (invocación) 28*
   4. *WSDL (descripción) 33*
   5. *Ejemplos 37*
6. ARQUITECTURA REST 38
   3. *Servicios web RESTful 42*
   2. *OAuth 2.0 ..... 50*
   3. *JWT 50*
   4. *Especificación de APIs: OpenAPI 52*
   5. *Ejemplos servicios web REST 52*
   6. *Códigos de estado HTTP 53*
   7. *Servicios web RESTful vs servicios web SOAP 54*
7. FRAMEWORKS 55
8. ARQUITECTURA HEXAGONAL 56

1. ARQUITECTURAS DE SISTEMAS C/S Y MULTICAPA

La arquitectura del software hace referencia a la estructuración del software (vista lógica) en capas, inicialmente, en una única máquina y, posteriormente, en varias máquinas (vista física), denominándose niveles.[1]

1. Componentes

Las capas genéricas a distribuir en los niveles físicos son las siguientes:

- **Capa de PRESENTACIÓN**: se encarga de presentar al usuario la información e interactúa con él. Está formada por los componentes de interfaz de usuario.

- **Capa de LÓGICA DE NEGOCIO**: formada por los objetos de negocio. Encargada de procesar y dar respuesta a las solicitudes de información de los usuarios, de comunicarse con las bases de datos, gestores documentales, otros servicios de la organización y de la publicación de servicios para ser consumidos por otros sistemas. Esta capa incluye la subcapa de persistencia o de acceso a datos.

- **Capa de DATOS**: responsable del almacenamiento y gestión de la información.

Las características más destacables que proporcionan las arquitecturas basadas en sistemas distribuidos, como son la arquitectura cliente/servidor, se enumeran a continuación:

- **Recursos compartidos**: un servidor puede atender a muchos clientes al mismo tiempo y regular sus accesos a los recursos compartidos.

- **Protocolos asimétricos**: hay una relación muchos a uno entre los clientes y un servidor. Los clientes siempre inician un diálogo mediante la solicitud de un servicio. Los servidores esperan pasivamente por las solicitudes de los clientes.

- **Separación de funcionalidad**: se divide entre cliente y servidor.

- **Código reutilizable**: la implementación de un servicio puede utilizarse en varios servidores.

- **Independencia**: el software de un sistema distribuido es independiente del hardware y del sistema operativo.

- **Integridad**: el código y los datos del servidor se mantienen centralizados, con lo que es menos costoso su mantenimiento y controlar la integridad de los datos compartidos.

[1] En inglés existe la diferencia entre layer (capa) y tier (nivel). Sin embargo, en español, se **suelen** traducir ambas como capa, lo que da lugar a confusión.

- **Escalabilidad**: los sistemas cliente/servidor se pueden escalar horizontal y verticalmente. Horizontalmente significa añadir o eliminar clientes. Verticalmente significa migrar a máquinas servidoras más potentes o multiservidores.

- **Interoperabilidad**: permite que los sistemas cliente/servidor puedan operar entre sí a pesar de que los sistemas sean de diferentes plataformas.

- **Transparencia**: consiste en la ocultación al usuario de la separación de los componentes en un sistema distribuido, de forma que se perciba el sistema como un todo más que como una colección de componentes independientes. Tipos de transparencia:
  - Transparencia de **acceso**: permite accede a los recursos locales y remotos empleando operaciones idénticas.
  - Transparencia de **localización (ubicación)**: El servidor es un proceso que puede residir en la misma máquina que los clientes o en una máquina diferente en una red. El software cliente/servidor oculta la localización del servidor a los clientes, redirigiendo las llamadas a los servicios cuando sea necesario. También permite acceder a los recursos sin conocer su localización.
  - Transparencia de **concurrencia**: un recurso puede ser compartido por varios usuarios, incluso cuando éstos compiten por él.
  - Transparencia de **replicación**: permite utilizar múltiples ejemplares de cada recurso para aumentar la fiabilidad y las prestaciones sin que los usuarios necesiten su conocimiento.
  - Transparencia frente **a fallos**: oculta al usuario los fallos permitiendo que las tareas puedan completarse y la recuperación de los recursos.
  - Transparencia de **movilidad (migración)**: permite la reubicación de los recursos a otras ubicaciones sin que afecte a las operaciones.
  - Transparencia de **prestaciones**: permite reconfigurar el sistema para mejorar las prestaciones según varía su carga.
  - Transparencia de **escalabilidad**: permite el escalado sin cambiar la estructura del sistema.

- **Encapsulación de servicios**: el servidor es un especialista, cuando se le entrega un mensaje solicitando un servicio, él determina cómo conseguir hacer el trabajo. Los servidores se pueden actualizar sin afectar a los clientes en tanto que la interfaz pública de mensajes que se utilice por ambos lados permanezca inalterable.

- **Intercambios basados en mensajes**: los clientes y servidores son procesos débilmente acoplados que pueden intercambiar solicitudes de servicios y respuestas utilizando mensajes (por ejemplo, llamadas a procedimiento remoto RPC).

La arquitectura física de un sistema está definida por el número de niveles que la componen. En función de eso, vamos a ver las diferentes arquitecturas.

2. Arquitectura mononivel o de 1 nivel

Todas las capas se alojan en una máquina o nivel, en el servidor. Los múltiples usuarios están conectados al servidor mediante "terminales tontos" que únicamente muestran en pantalla la actividad que está realizando el servidor (un mainframe). La aplicación es **monolítica** y **centralizada**. El servidor es el responsable de todas las capas de la aplicación.

Se trata de una arquitectura obsoleta, propia de los años 80. En aquella época la mayoría de las aplicaciones multiusuario tenían esta arquitectura. Los mainframes soportaban un número limitado de terminales, ocupadas por usuarios de la empresa que introducían datos y solicitaban listados.

La única forma de escalar esta arquitectura (aumentar sus prestaciones cuando es necesario aumentar el número de usuarios) es migrar la aplicación a un mainframe más potente (y comprar más terminales).

La distribución de capas (vista lógica) en la arquitectura mononivel nos ofrece una única configuración posible (relleno en negro indica que las capa que se ubican en el mainframe):

Capas		
Presentación	**Lógica de negocio**	**Datos**

Configuración única

Ventajas:

- **Eficiencia**: al no existir varios niveles se elimina la comunicación entre ellos.
- **Facilidad de instalación y despliegue**.

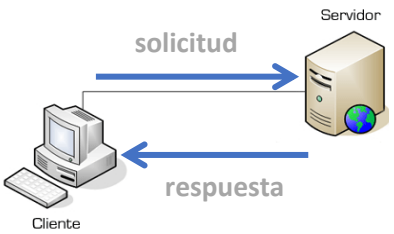
Desventajas:

- **Mantenimiento costoso y de complicada realización**: dificulta la incorporación de nuevas funcionalidades y la corrección de errores al ser un sistema monolítico y existir interdependencias en los módulos (alto acoplamiento).
- **Arquitectura poco escalable**.
- **Bajo rendimiento**.

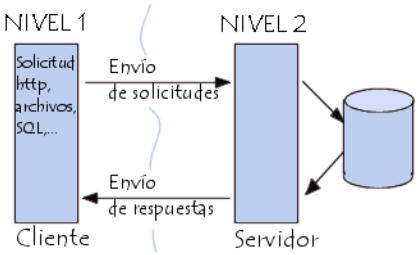
3. Arquitectura cliente/servidor o de 2 niveles

El paradigma cliente/servidor define una arquitectura que divide el proceso de la aplicación en dos procesos diferentes, ubicando, normalmente, la ejecución de cada proceso en 2 máquinas.

Esta arquitectura se conoce también como arquitectura de 2 niveles. El término 2 niveles describe la manera en que el procesamiento de la aplicación puede dividirse en dos procesos, **cliente** y **servidor**.



Los elementos de esta arquitectura son:



- **Cliente (front-end)**: realiza peticiones al servidor, es decir, consume servicios. Puede ser un navegador web o una app de escritorio. Es el nivel 1.

- **Servidor (back-end)**: procesa y responde las peticiones de los clientes, es decir, provee servicios. Normalmente se implemente mediante un servidor web (Apache, Tomcat) o un SGBD o ambos en un mismo servidor. Es el nivel 2.

- **Protocolos de comunicaciones**: interconectan los elementos anteriores.

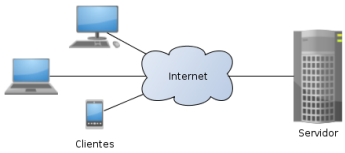
La distribución de capas (vista lógica) en la arquitectura cliente/servidor nos posibilita las siguientes configuraciones (relleno negro indica que la capa o parte de ella se ubica en el cliente):

Capas			
Presentación	**Lógica de negocio**	**Datos**	
■			Presentación distribuida o embellecida
■			Presentación remota o cliente ligero
■	■		Lógica distribuida
■	■		Datos remotos o cliente pesado
■	■	■	Datos distribuidos

- **Presentación distribuida**: la presentación se distribuye entre el cliente y el servidor. El servidor se encarga de la lógica de negocio y los datos.
- **Presentación remota**: el cliente es responsable de la presentación (presentación de información). El servidor se encarga de la lógica de negocio y los datos. Esta configuración también es conocida como cliente ligero. Como ejemplo están las apps de escritorio cuya única misión es presentar una interfaz con el usuario y se conectan a un servidor para trabajar.
- **Lógica distribuida**: el cliente se encarga de la presentación. La lógica de negocio es compartida entre el cliente (validación de datos de entrada, formato de los datos de salida…) y el servidor (resto). Los datos son responsabilidad del servidor. Por ejemplo, una aplicación web que realiza mediante Javascript algún tipo de procesamiento de negocio y el resto se hace en el lado del servidor mediante JSP o servlets.
- **Datos remotos**: de la presentación y lógica de negocio se encarga el cliente, los datos son gestionados por el servidor. Son aplicaciones monolíticas poco adaptables a cambios y poco escalables a nuevas necesidades. Esta configuración también es conocida como cliente pesado.

- **Datos distribuidos**: el cliente se encarga de la presentación y la lógica de negocio. La capa de datos es compartida entre cliente y servidor.

La mayoría de las **aplicaciones web** han sido aplicaciones de dos niveles pues proporcionan una interfaz común para acceder a la información a través de Internet. Utilizando como ejemplo la web: el navegador web que reside en el cliente solicita los datos (páginas HTML) almacenadas en el servidor web.



Hoy en día, la mayor parte de las aplicaciones web tienen unas necesidades y una complejidad que han sobrepasado la funcionalidad que otorga este tipo de arquitectura de 2 niveles.

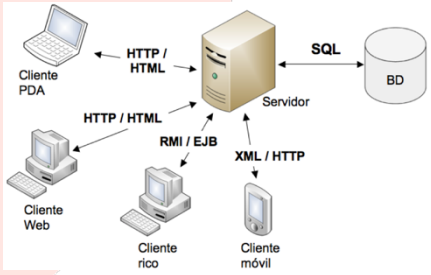
Ventajas:

- Se puede aprovechar las capacidades de cómputo del cliente.
- **Escalabilidad mejorada** respecto a la arquitectura mononivel.
- Permite personalizar la capa de presentación para distintos fines y portarla a distintos entornos (multiplataforma).
- Eficiencia en el lado del servidor.

Inconvenientes:

- **Protocolos más complejos** y gestión de sesiones complican la escalabilidad.
- **Arquitectura inadecuada** cuando se necesita integrar más de un servidor.
- **Mantenimiento costoso** debido a que cualquier cambio en la lógica de negocio o de presentación supone una reinstalación en todos los clientes.

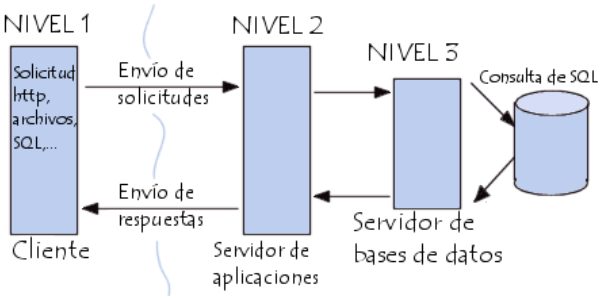
4. Arquitectura de 3 niveles



En la arquitectura en tres niveles aparecen tres funciones bien diferenciadas: el almacenamiento y gestión de los datos, el proceso de la lógica de la aplicación y la presentación de los datos e interacción con el usuario.

Los elementos de esta arquitectura son:

- **Cliente**: realiza peticiones al servidor, es decir, consume servicios. Puede ser un navegador web o una app de escritorio/móvil. Muestra el resultado de la capa de presentación mediante el navegador web o se encarga de la interfaz de usuario (y, por tanto, reside la capa de presentación) si se opta por app de escritorio/móvil. Es el nivel 1. Será el front-end si alberga la capa de presentación.



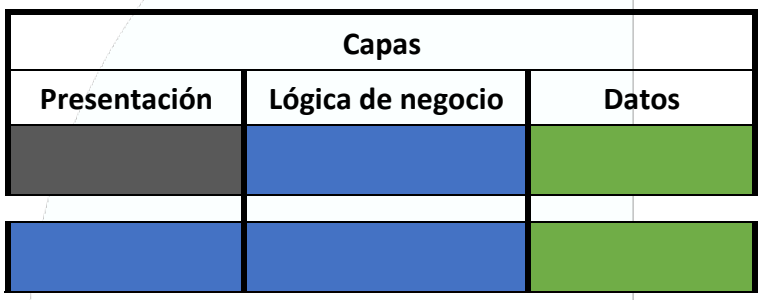
- **Servidor web/aplicaciones**: procesa y responde las peticiones de los clientes, es decir, provee servicios. Normalmente se implementa mediante un servidor

web (Apache, Tomcat) o un servidor de aplicaciones que incorpora un servidor web. Contiene la capa de presentación (si servidor web) y la capa de negocio. Es el nivel 2. Será el front-end si existe un servidor web que albergue la capa de presentación. El servidor de aplicaciones será en todo caso el back-end.

- **Servidor de datos (SGBD)**: procesa y responde las peticiones de datos del nivel anterior. Contiene la capa de datos. Es el nivel 3.

- **Protocolos de comunicaciones**: interconectan los elementos anteriores.

La distribución de capas (vista lógica) en la arquitectura de 3 niveles nos posibilita las siguientes configuraciones (relleno negro indica que la capa se ubica en el nivel 1 cliente, relleno azul indica que la capa se ubica en el servidor de web/aplicaciones en el nivel 2, relleno verde especifica que la capa de datos se ubica en el servidor de datos en el nivel 3):



App de escritorio/app móvil + servidor de aplicaciones + servidor de datos

Navegador web + servidor de aplicaciones + servidor de datos

La capa de presentación es completamente independiente de la localización, acceso y manipulación de los datos, pues el acceso a datos corresponde a la capa de aplicación y la manipulación de estos es tarea del propio sistema gestor de base de datos.

Asimismo, los objetos de la aplicación son completamente independientes de la presentación, permitiendo a la vez ser mostrados por una aplicación cliente estándar o un navegador. Este hecho favorece la reutilización de los objetos, pues su independencia de la presentación garantiza la validez de su proceso para cualquier aplicación.

Por último, se ofrece el aislamiento de la interfaz de los datos a los objetos externos. Dichos objetos conocen qué datos obtienen, pero no cómo los obtienen, pues los objetos de la capa intermedia se encargan de ocultarlo, ofreciendo solamente la interfaz de acceso a datos.

A nivel físico, los clientes se hacen más finos (thin client) y se estandarizan con algún lenguaje (y aplicación) de presentación, como HTML (browser) o app de escritorio/móvil. Existen dos servidores, uno que gestiona la capa de datos y otro que genera la vista y se la envía a los clientes.

La arquitectura ideal para las aplicaciones web es esta arquitectura en 3 niveles.

Ventajas:

- **Reutilización y ampliación**: el hecho de separar la capa de datos de la capa de lógica de negocio hace que la aplicación sea más fácil de ampliar y reusar.

- **Mantenimiento más sencillo**: no es necesario reinstalar los clientes cuando se modifica la aplicación y por el bajo acoplamiento entre funcionalidades.
- **Mejora la estructuración de la aplicación y el encapsulamiento.**
- **Mayor escalabilidad.**
- **Incremento del rendimiento**.

Inconvenientes:

- **Aumento de la complejidad**: al aumentar el número de servidores.
- **Mayor coste económico**: se requieren 2 servidores.

5. Arquitectura multicapa y arquitecturas de 3 niveles y multinivel

El objetivo inicial de alcanzar flexibilidad, acceso rápido y seguro a los datos corporativos, a un precio razonable, seguía siendo una meta por conseguir. El coste de gestión de estos sistemas era enorme y las corporaciones seguían buscando una nueva tecnología que resolviese al menos algunos de estos problemas.

A finales de los 90, gran número de desarrollos corporativos empezaron a aprovechar los estándares, lenguajes y protocolos surgidos alrededor de la Web, creándose el concepto de las **arquitecturas multicapa**, que se presentaban como alternativa a la arquitectura cliente/servidor clásica, que permitiría utilizar un entorno cliente ligero y, por lo tanto, evitar el problema de la distribución y mantenimiento del software de los PCs cliente.

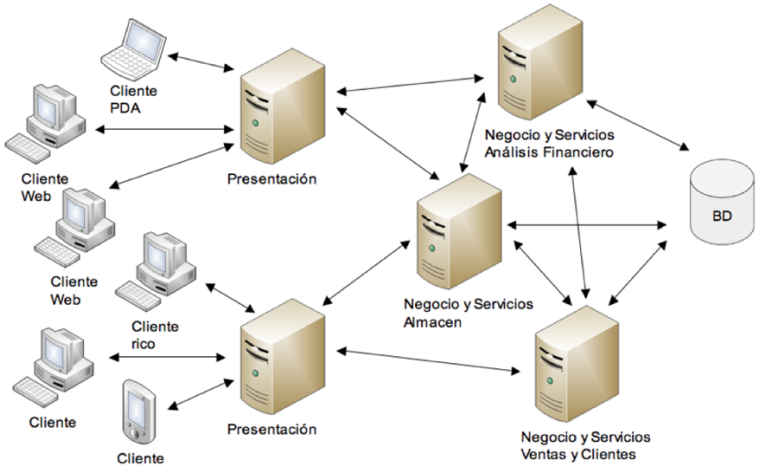
Indudablemente estos inicios fueron duros ya que la tecnología que se estaba utilizando no había sido inicialmente pensada para lo que ahora se quería utilizar. Las empresas de software iniciaron una carrera para conseguir adaptar los protocolos, lenguajes y estándares de Internet, que parecían claramente asentados en un tipo de aplicaciones con gran cantidad de información estática e interfaces de usuario sencillas, a las necesidades de los desarrollos corporativos estratégicos. Java es uno de estos lenguajes que aparecieron alrededor de Internet. Inicialmente, los applets de Java contribuyeron a enriquecer las interfaces de usuario, posteriormente, tecnologías como los servlets y los JSPs, facilitaron el desarrollo de la lógica de negocio compleja para los servidores. Podríamos decir que Java ha impulsado el desarrollo de aplicaciones con arquitectura multicapa como ningún otro lenguaje.

Hasta ahora hemos descrito las arquitecturas en función del número de niveles físicos, teniendo en cuenta que el número de capas eran invariable: 3. Por lo que las distintas arquitecturas nos permitían la ubicación de esas 3 capas ya preestablecidas.

Sin embargo, en general, la capa de lógica de negocio suele ser subdivida en varias capas. De ahí que se disponga de un número de capas lógicas superior a 3. Los sistemas actuales siguen

esta arquitectura. Además, se cumple con el artículo 7 del ENI (Esquema Nacional de Interoperabilidad)[2].

Este tipo de arquitectura es el que da lugar a las arquitecturas multinivel, ya que las capas se suelen ubicar en 3 o más niveles. Por tanto, las arquitecturas multicapa utilizan arquitecturas físicas de 3 niveles o multinivel.

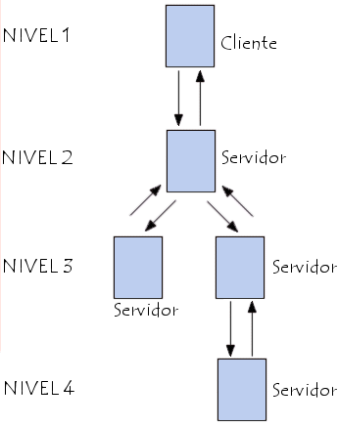


Los elementos de esta arquitectura son:

- **Cliente**: realiza peticiones al siguiente nivel, es decir, consume servicios. Suele ser un navegador web, mostrando el resultado de la capa de presentación. Es el nivel 1.

- **Servidor web**: encargada de la presentación a los clientes y de dirigir las peticiones al servidor de aplicaciones correspondiente. Implementado mediante servidor web. Podemos tener varios servidores web balanceados, denominados frontales web. Es el nivel 2. Es el front-end.

- **Servidores de aplicaciones**: procesan y responden las peticiones del nivel anterior. Podemos tener un servidor o varios si cada uno gestiona una parte del sistema (por ejemplo, recursos humanos, contabilidad y servicios al cliente). Contiene la capa de negocio. Es el nivel 3. Es el back-end.



- **Servidor de datos (SGBD) y servidores documentales**: procesan y responden las peticiones de datos/documentos del nivel anterior. Contiene la capa de datos. Es el nivel 4.

- **Protocolos de comunicaciones**: interconectan los elementos anteriores.

[2] **Enfoque de soluciones multilaterales**. Se favorecerá la aproximación multilateral a la interoperabilidad de forma que se puedan obtener las ventajas derivadas del escalado, de la aplicación de las arquitecturas modulares y multiplataforma, de compartir, de reutilizar y de colaborar.

La distribución de capas (vista lógica) en la arquitectura de multinivel nos posibilita la siguiente configuración (sin relleno significa que no tiene ninguna responsabilidad como es el nivel 1, relleno negro indica que la capa se ubica en el nivel 2 servidor web, relleno azul indica que la capa se ubica en los servidores de aplicaciones en el nivel 3, relleno verde especifica que la capa de datos se ubica en el servidor de datos/documentos en el nivel 4):

Cliente	Capas			
	Presentación	Lógica de negocio	Datos	
NIVEL 1	NIVEL 2	NIVEL 3 (varios servidores)	NIVEL 4	Navegador + servidor web + servidor(es) de aplicaciones + servidor de datos/documentos
NIVEL 1	NIVEL 2 (interfaces)	NIVEL 3 (varios servidores)	NIVEL 4	App de escritorio/app móvil o servidor web + servidor(es) de aplicaciones + servidor de datos/documentos

La capa de lógica de negocio se suele dividir en:

- **Capa de negocio**: puede dividirse a su vez en distintas áreas del negocio.
- **Capa de seguridad**: encargada de la autenticación y autorización de usuarios y sistemas.
- **Capa de Business Intelligence**: cuadro de mando destinado a la explotación estadística.
- **Capa de infraestructura**: para consumo de servicios comunes.
- **Capa de servicios**: dedicada a la publicación de servicios disponible para otros sistemas.
- **Capa de persistencia**: acceso a bases de datos, gestores documentales y otras fuentes de datos.

Ventajas:

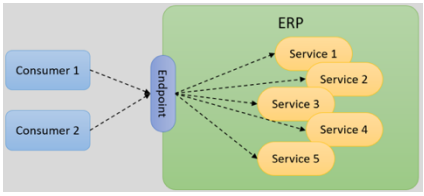
- Mejora el **escalado**.
- **Compartición y reutilización**.
- La separación de roles en capas hace más fácil **modificar y reemplazar capas**.
- Facilita el mantenimiento.
- Bajo acoplamiento y alta cohesión.
- Mejora del rendimiento.
- El **aumento del tamaño o complejidad** de una capa no afecta a las demás.

Inconvenientes:

- Incremento del tráfico en la red.
- Aumento de la complejidad del sistema.
- Incremento del coste.

2. ARQUITECTURA ORIENTADA A SERVICIOS (SOA)

Según OASIS[3], se puede definir **SOA** (**Service Oriented Architecture**, Arquitectura Orientada a Servicios) como un paradigma para organizar y utilizar capacidades **distribuidas** y bajo el control de diferentes propietarios y dominios. Provee una manera uniforme de ofrecer, descubrir, interactuar y usar dichas capacidades para producir los efectos deseados de manera consistente y medible.



El objetivo principal de la arquitectura SOA es ofrecer una estrategia para la **integración** de aplicaciones, enfocada en la **construcción de servicios y no de aplicaciones finales**.

Un servicio expondrá una funcionalidad definida, mientras que la aplicación solamente será la encargada de orquestar la ejecución de un conjunto de servicios. Estos servicios podrán ser, a su vez, reutilizados en otras aplicaciones.

Así, la arquitectura definirá:

- Servicios a utilizar.
- Interacciones entre servicios.
- Tecnologías con que se implementarán.
- Interfaces entre servicios (mensajes soportados, contenidos, etc).

Manifiesto SOA

- La Orientación a Servicios es un paradigma que enmarca lo que usted hace. SOA es un tipo de arquitectura que resulta de aplicar la orientación a servicios.
- Hemos estado aplicando la orientación a servicios para ayudar a las organizaciones a entregar consistentemente valor a los negocios, de manera sostenida, con mayor agilidad y con efectividad en los costes, alineada con las necesidades cambiantes de los negocios.
- A través de nuestro trabajo hemos llegado a priorizar:
  - **El Valor del Negocio** por encima de la estrategia técnica.
  - **Las Metas Estratégicas** por encima de los beneficios específicos de los proyectos.
  - **La Interoperabilidad Intrínseca** por encima de la integración personalizada.
  - **Los Servicios Compartidos** por encima de las implementaciones de propósito específico.
  - **La Flexibilidad** por encima de la optimización.
  - **El Refinamiento Evolutivo** encima de la búsqueda de la perfección inicial.
- Esto significa que, valoran más los elementos de la izquierda que los de la derecha.

[3] **OASIS** (*Organization for the Advancement of Structured Information Standards, Organización para el Avance de Estándares de Información Estructurada*), es un consorcio internacional sin ánimo de lucro que se orienta al desarrollo, la convergencia y la adopción de los estándares de comercio electrónico y servicios web.

El diseño orientado a servicios, base de esta arquitectura, cuenta con **8 principios de diseño**:

(1) **INTEROPERABILIDAD (Contrato de servicio)**: descripción de los servicios cumpliendo los mismos estándares de diseño. Cualquier sistema de cliente puede ejecutar un servicio, independientemente de la plataforma o el lenguaje de programación.

(2) **ACOPLAMIENTO FLEXIBLE (Bajo acoplamiento)**: evitando acoplarse a la tecnología que los implementa y minimizando la dependencia entre servicios.

(3) **ABSTRACCIÓN**: presentando la información mínima requerida (la interfaz), ocultan la lógica (implementación) al resto de servicios. Pueden verse como una caja negra.

(4) **Reutilización**.

(5) **Autonomía**.

(6) **Ausencia estado**: delegando el manejo de estados en la aplicación.

(7) Garantizan su **descubrimiento**.

(8) Preparados para ser utilizado en **composiciones** (orquestados por la aplicación).

Las capas de la arquitectura SOA son:

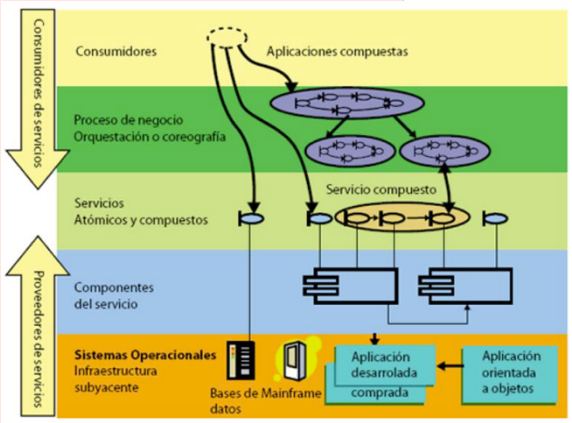
- La capa de **sistemas operacionales** está constituida por sistemas legacy, sistemas orientados a objetos, y aplicaciones de BI, que podrán ser integrados utilizando técnicas de integración orientadas a servicios.

- En la **capa de coreografía/coreografía** se establece el flujo para que los servicios actúen conjuntamente ofreciendo el resultado como si fuera una única aplicación.

- La **capa de servicios** permite que estos servicios sean expuestos y descubiertos para ser invocados por las aplicaciones.

- Los **componentes de servicio** aseguran los acuerdos a nivel de servicio y la calidad de los servicios externos. Está soportada por los servidores de aplicaciones que ofrecen alta disponibilidad, balanceo de carga, etc.

- Los **consumidores de servicios** son aquellos procesos que hacen uso de los servicios del sistema.



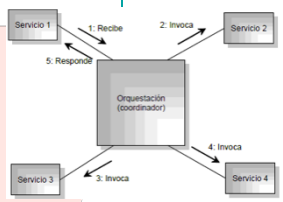
Existirán adicionalmente las capas transversales de integración y calidad del servicio y monitorización, encargados de las tareas de seguridad, disponibilidad, etc.

Esto nos lleva a definir los 4 componentes principales de la arquitectura SOA:

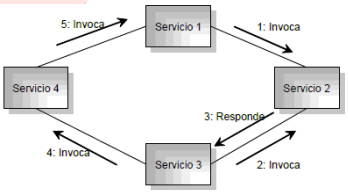
- **SERVICIO**: son los componentes básicos. Pueden ser privados (para usuarios internos) o públicos (accesibles para todos desde Internet). Cada servicio tiene 3 características fundamentales:
  - IMPLEMENTACIÓN del servicio: es el código, la función específica para resolver un problema (por ejemplo, crear una solicitud o calcular el total de una factura).
  - CONTRATO del servicio: son las condiciones bajo las que se ofrece el servicio, como los prerrequisitos para su uso, el coste de uso o la calidad del servicio proporcionada.
  - INTERFAZ del servicio: es la exposición del servicio para ser consumida por las aplicaciones clientes. Puede verse como la definición de un método en una interface en Java.
- **PROVEEDOR de servicios**: indica el propietario del servicio.
- **CONSUMIDOR de servicios**: es el cliente que consume el servicio.
- **REGISTRO de servicios**: es un directorio de servicios donde se encuentran los servicios disponibles a consumir puestos a disposición por el proveedor junto con la documentación descriptiva de los mismos.

Colaboración entre servicios

- **ORQUESTACIÓN**: Un proceso se puede considerar una orquestación de servicios cuando es controlado totalmente por una única entidad. Este proceso define completamente las interacciones con los servicios componentes y la lógica requerida para conducir correctamente esas interacciones, y sólo la entidad que está orquestando el proceso conoce el flujo de control e información que sigue el proceso que se está orquestando.



- **COREOGRAFÍA**: Un proceso es una coreografía de servicios cuando define las colaboraciones entre cualquier tipo de aplicaciones componentes, independientemente del lenguaje o plataforma en el que estén definidas las mismas. Un proceso de coreografía no es controlado por uno solo de los participantes, y a diferencia de la orquestación, puede verse como un proceso público y no ejecutable. Es público porque define un comportamiento común que todas las entidades participantes deben conocer, y es no ejecutable porque está pensado para verse más bien como un protocolo de negocio que dicta las reglas para que dichas entidades puedan interactuar entre sí.

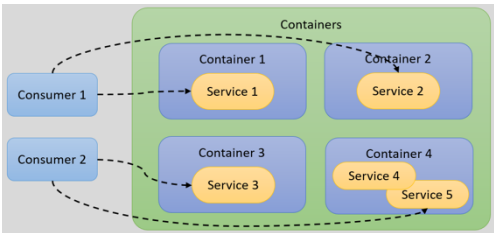


IMPORTANTE

SOA se puede construir sobre servicios web estándar, como por ejemplo SOAP, que gozan de gran aceptación en el mercado. Sin embargo, se puede utilizar cualquier tecnología basada en servicios. **SOA define QUÉ, los servicios web definen CÓMO**.

3. ARQUITECTURA DE MICROSERVICIOS

La arquitectura de microservicios, conocido por las siglas MSA (**Micro Services Architecture**) es una aproximación para el desarrollo de software que consiste en construir una aplicación como un conjunto de **pequeños** servicios (unidad mínima) **independientes**, los cuales se ejecutan en su propio proceso de forma independiente y se **comunican mediante mecanismos ligeros** (normalmente una API de recursos HTTP).





Cada servicio se encarga de implementar una funcionalidad completa del negocio y es desplegado de forma independiente (cada servicio en un proceso). Es posible que cada uno de estos servicios esté programado en distintos lenguajes y use diferentes tecnologías de almacenamiento de datos.

Los microservicios son un estilo o enfoque arquitectónico y organizativo que estructura una aplicación como una colección de servicios que son (características):

- Altamente mantenibles y comprobables.
- **Débilmente acoplados**.
- **Reutilizables**.
- **Autónomos**: cada servicio se puede desarrollar, implementar, operar y escalar sin afectar el funcionamiento de otros servicios. Los servicios no necesitan compartir ninguno de sus códigos o implementaciones con otros servicios. Cualquier comunicación entre componentes individuales ocurre a través de API bien definidas.
- **Desplegados de forma independiente**.
- **Especializados**: cada servicio está diseñado para un conjunto de capacidades y se enfoca en resolver un problema específico.
- Organizados en torno a capacidades empresariales.
- Permiten la implementación de aplicaciones grandes y complejas.

Ventajas del uso de microservicios:

- **Agilidad**: fomentan una organización de equipos pequeños e independientes que se desarrollan los servicios para trabajar de forma más independiente y más rápida. Esto acorta los tiempos del ciclo de desarrollo, obteniendo un beneficio significativo del aumento de rendimiento de la organización.
- **Escalado flexible**: permiten que cada servicio se escale de forma independiente para satisfacer la demanda de la característica de la aplicación que respalda. Esto permite a los equipos adecuarse a las necesidades de la infraestructura, medir con precisión el coste de una característica y mantener la disponibilidad si un servicio experimenta un aumento en la demanda.
- **Implementación sencilla**: permiten la integración y la entrega continua, lo que facilita probar nuevas ideas y revertirlas si algo no funciona. El bajo coste de los errores permite

experimentar, facilita la actualización del código y acelera el tiempo de comercialización de las nuevas características.

- **Libertad tecnológica**: las arquitecturas de microservicios no siguen un enfoque de "diseño único". Los equipos tienen la libertad de elegir la mejor herramienta/lenguaje/tecnología para resolver sus problemas específicos.

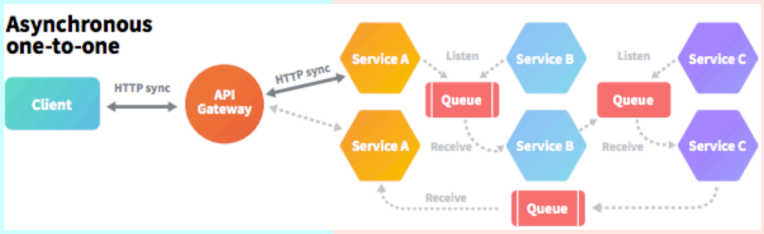
Comunicación entre microservicios

Un punto importante a resolver es la comunicación entre microservicios. Se plantean dos opciones:

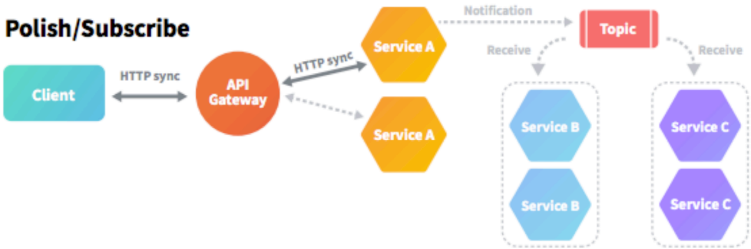
- **Comunicación SÍNCRONA**: (petición/respuesta) API REST sobre HTTP/HTTPS, SOAP, GraphQL, gRPC.



- **Comunicación ASÍNCRONA**: basada en mensajes o eventos y en el uso colas. Existen dos tipos de comunicación asíncrona:
  - **Un solo receptor (one-to-one):** basado en mensajes y colas. Comunicación 1 a 1 donde el servicio receptor puede tener una carga computacional alta y se recurre al uso de una cola para leer las peticiones pendientes, permitiendo al servicio emisor no esperar (no bloquearse). Así pues emisor y receptor pueden trabajar a ritmos distintos. Desde el punto de vista funcional, los mensajes son procesados por el servicio receptor una única vez y eliminados de la cola una vez tratados. Se sigue el patrón Productor-Consumidor.



  - **Varios receptores (publish/subscribe):** basado en eventos y bus. Múltiples receptores pueden procesar las peticiones de un bus. Los servicios cliente publican en el bus eventos para que sean gestionados por los servicios que se han suscrito a ese evento.



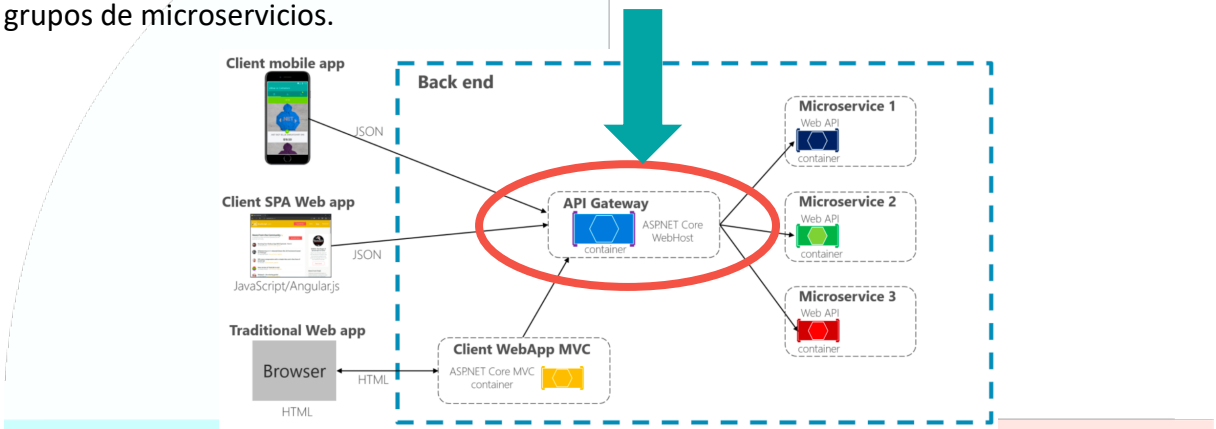
Se suele utilizar el protocolo de comunicación **AMQP** (Advanced Message Queuing Protocol). Es un protocolo de cola de mensajes de la capa de aplicación y de código abierto

que se encarga de la transmisión de mensajes, el almacenamiento de mensajes en una cola y la distribución de mensajes.

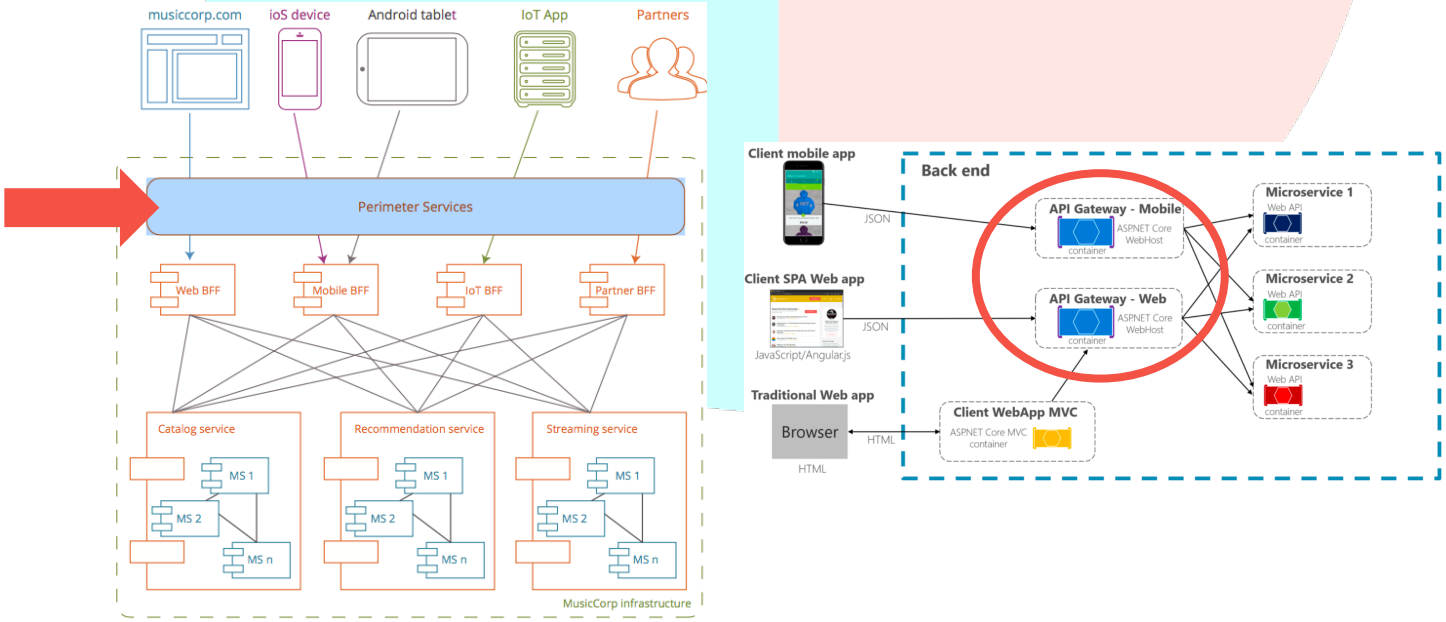
Se usará un **BRÓKER DE MENSAJERÍA** que se encarga de la distribución del mensaje desde el buzón a diferentes colas, de acuerdo con unas reglas previamente definidas. Permite que las aplicaciones consumidoras (publicadores) estén desacopladas de los servicios. Destacan las soluciones **RabbitMQ** y **Apache Kafka**.

API Gateway

Cuando se diseñan y construyen aplicaciones basadas en microservicios, grandes o complejas, y con múltiples tipos de clientes, un buen enfoque a considerar es un API Gateway, el cual ofrece un servicio como único punto de entrada (como el patrón Fachada) para todos los grupos de microservicios.



Puede ocurrir que se diseñe una solución con varios API Gateways, conocido como patrón Backend For Frontend (BFF), donde cada API GW ofrece servicios a grupo determinado de microservicios. Se ilustra en las siguientes imágenes:



El API Gateway atiende las solicitudes de los diferentes tipos de solicitudes, redirigiéndolas al microservicio adecuado.

Es un proxy inverso que permite **administrar, configurar y enrutar** las solicitudes a las API de la aplicación. Proporciona una interfaz API REST (u otras como WebSocket) para hacer de puerta de enlace hacia un grupo de microservicios.

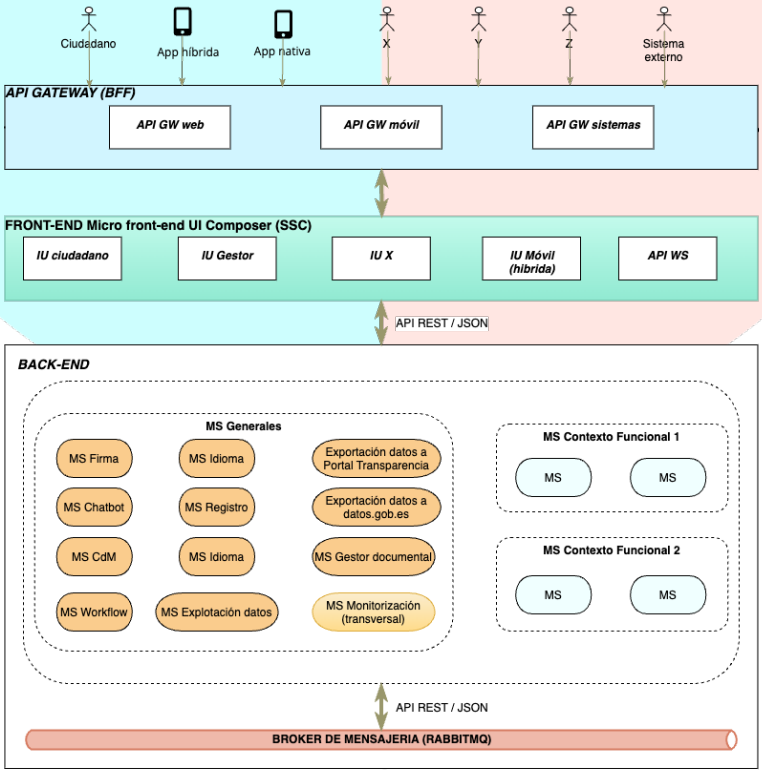
Además de las funciones básicas, un API Gateway puede encargarse de:

- La autenticación de los clientes.
- La validación de los datos de entrada.
- La transformación de la respuesta dada por un microservicio para ser entregada al cliente en el formato esperado. Por ejemplo, el microservicio responde mediante un XML y el API Gateway lo transforma a un JSON para entregarlo al cliente.
- Gestión del tráfico.
- La recopilación de mediciones y estadísticas sobre solicitudes.

Soluciones:

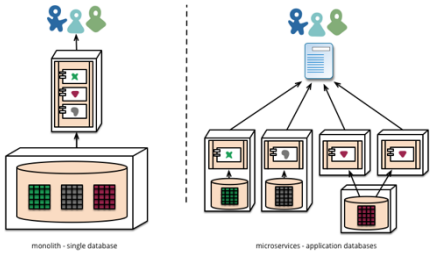
- **Kong** (de código abierto).
- Amazon API Gateway.
- Azure API Management.
- RedHat 3scale.
- MuleSoft.
- WSO2.
- Google APIgee.

Esquema de aplicación basada en microservicios



Arquitectura monolítica vs Arquitectura de microservicios

Como podemos observar en la imagen, mientras que la arquitectura monolítica consta de pocos componentes, la arquitectura de microservicios está formada por un número muy superior. Esto se crea para solucionar el problema de la visión monolítica de interdependencia, desacoplando componentes hacemos que estos sean independientes entre ellos.



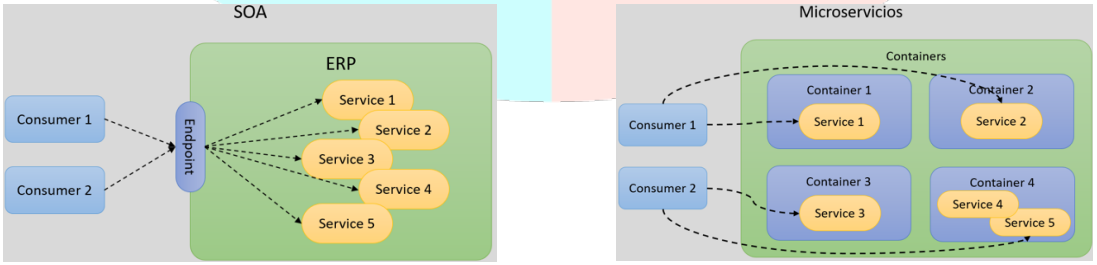
Ventajas:

- Cada microservicio se puede desplegar de forma **independiente**: un cambio en un módulo no afectará a los demás, basta con subir ese módulo.
- Es fácil de entender, ya que la lógica de negocio está bien separada.
- Facilita la gestión de equipos **multifuncionales y autónomos**. Por sí mismo, cada microservicio es multifuncional: tiene una parte de base de datos, de backend, además de ser independiente de los demás. Podemos formar equipos multifuncionales que se encarguen de varios microservicios, escalando el proceso de desarrollo de forma más sencilla.
- Es más fácil de **escalar** a nivel de software, ya que en lugar de replicar toda la aplicación y gestionarlo con balanceadores, replicaremos los microservicios que tengan más carga.

Inconvenientes:

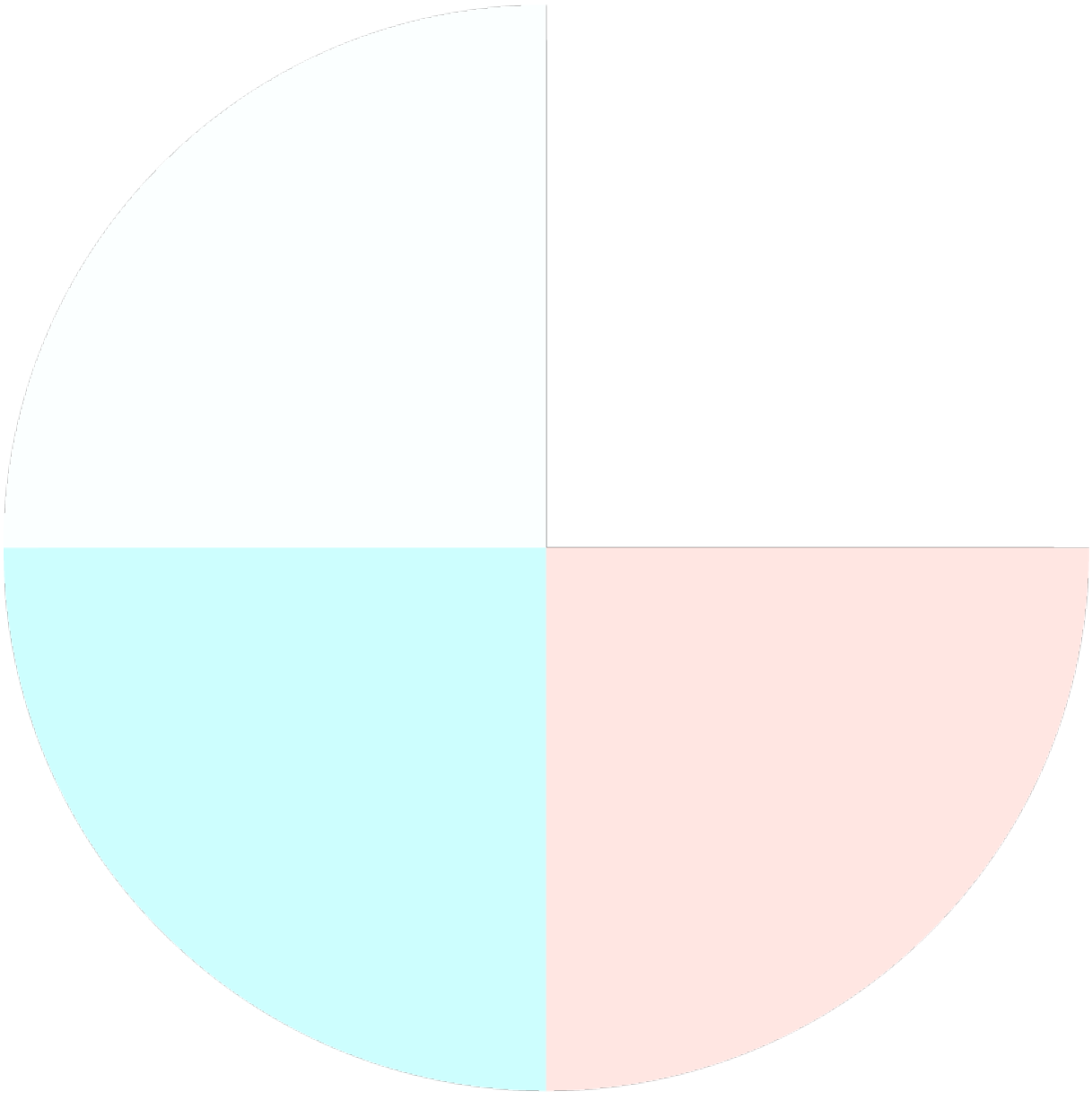
- Los microservicios introducen **complejidad**. Hay que hacer un gran esfuerzo en despliegues automáticos (si tenemos 300 microservicios no puedes estar desplegando cada uno de ellos a mano), monitorización (si falla uno, no es buena práctica ir mirando uno a uno a ver qué pasa), métricas, trazabilidad de peticiones, seguridad, gestión de fallos, consistencia de datos, estrategia de pruebas y más factores que introducen los sistemas distribuidos.
- La **comunicación** entre microservicios se realizaba punto a punto y se dotaba a cada microservicio de "inteligencia adicional". En la actualidad, algunos de estos aspectos (el descubrimiento de servicios) ya son proporcionados por algunas de las plataformas de orquestación (Openshift o Kubernetes) pero aún no los cubren en su totalidad.

Arquitectura SOA vs Arquitectura de microservicios



SOA proporciona un enfoque arquitectónico para toda una organización mientras que los **microservicios** son una estrategia de implementación para una aplicación que utilizan los equipos de desarrollo de software.

Los microservicios además se consideran una evolución de la arquitectura SOA, la cual nos invita a crear aplicaciones más modulares, capaces de funcionar de una forma autónoma y que puedan ser reutilizadas de una forma más eficaz. De la misma manera, optimiza la utilización del hardware, pues solo son desplegados los servicios necesarios, en lugar de desplegar aplicaciones completas.



4. ARQUITECTURA DE SERVICIOS WEB

Un **servicio web** (WS) es un servicio, con un interfaz definida y conocida, que expone funcionalidad de un sistema de información mediante tecnologías estándar web.

La iniciativa UDDI[4] define un WS como aplicaciones de negocio modulares y autocontenidas que tienen interfaces **abiertos**, orientados a **Internet** y basado en **interfaces estándar**.

Esta definición requiere interfaces abiertos, es decir, que han de ser públicos y pueden ser invocados a través de Internet. También indica que los WS han de ser autocontenidos, esto significa que deben poseer toda información para poder ser consumidos.

Por su parte, el W3C define un WS como un sistema software diseñado para soportar la **interacción máquina-a-máquina** a través de una red y de forma interoperable. Normalmente, cuenta con una interfaz descrita en una interfaz procesable por un sistema (normalmente WSDL) a través de la que es posible interactuar mediante mensajes SOAP transmitidos usando serialización XML sobre HTTP (normalmente) juntamente con otros estándares web.

Usualmente nos referimos con WS a una colección de métodos a los que podemos llamar desde cualquier lugar de Internet o de nuestra intranet, siendo este mecanismo de invocación totalmente independiente de la plataforma que utilicemos y del lenguaje de programación en el que se haya implementado internamente el servicio.

Los WS constituyen el siguiente paso en la evolución de la tecnología orientada a objetos, y representan una revolución al alejarse de las arquitecturas tradicionales tipo cliente-servidor a nuevas arquitecturas distribuidas tipo igual-a-igual (peer-to-peer). Estos servicios consisten en un conjunto de estándares que permiten a los desarrolladores implementar aplicaciones distribuidas, utilizando herramientas muy distintas para crear aplicaciones que utilizan una combinación de módulos de software que son llamados desde diversos sistemas distribuidos en regiones geográficas distintas.

1. Tipos de servicios web

A nivel técnico, los servicios pueden implementarse de varias formas. En este sentido, podemos distinguir dos tipos de WS: los denominados servicios Web "grandes" ("big" Web Services), denominados servicios Web SOAP y los servicios Web RESTful.

[4] UDDI es una iniciativa dirigida por distintos fabricantes de software, dentro el consorcio de estándares OASIS que propone una plataforma estándar e interoperable que permite a las aplicaciones de forma sencilla y dinámica, encontrar y utilizar servicios Web sobre Internet.

OASIS (http://www.oasis-open.org) dirige el desarrollo, la convergencia y la adopción de estándares para el comercio electrónico.

(1) Servicios Web SOAP

Los servicios Web SOAP, o servicios Web "big", utilizan mensajes XML para intercomunicarse que siguen el estándar **SOAP** (**Simple Object Access Protocol**), un lenguaje XML que define la arquitectura y formato de los mensajes. Dichos sistemas normalmente contienen una descripción legible por la máquina de la descripción de las operaciones ofrecidas por el servicio, escrita en WSDL (Web Services Description Language), que es un lenguaje basado en XML para definir las interfaces sintácticamente.

El formato de mensaje SOAP y el lenguaje de definición de interfaces WSDL se ha extendido bastante, y muchas herramientas de desarrollo integrado (IDE) pueden reducir la complejidad de desarrollar aplicaciones de servicios web.

El diseño de un servicio basado en SOAP debe establecer un contrato formal para describir la interfaz que ofrece el servicio web. WSDL puede utilizarse para describir los detalles del contrato, que pueden incluir mensajes, operaciones, bindings, y la localización del servicio web. También deben tenerse en cuenta los requerimientos no funcionales, como por ejemplo las transacciones, necesidad de mantener el estado (addressing), seguridad y coordinación.

(2) Servicios Web RESTful

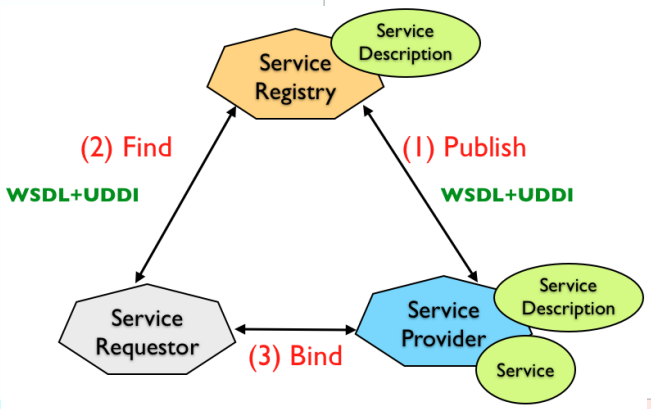
Los servicios Web **RESTful** (**Representational State Transfer Web Services**) son adecuados para escenarios de integración básicos ad-hoc. Dichos servicios web se suelen integrar mejor con HTTP que los servicios basados en SOAP, ya que no requieren mensajes XML o definiciones del servicio en forma de fichero WSDL.

Los servicios Web REST utilizan estándares muy conocidos como HTTP y URI y tienen una infraestructura "ligera" que permite que los servicios se construyan utilizando herramientas de forma mínima. Gracias a ello, el desarrollo de servicios RESTful es barato y tiene muy pocas "barreras" para su adopción.

5. SERVICIOS WEB SOAP

1. Arquitectura

Los servicios web presentan una Arquitectura Orientada a Servicios (**SOA**) que permite crear una definición abstracta de un servicio, proporcionar una implementación concreta de dicho servicio, publicar y localizar un servicio, seleccionar una instancia de un servicio y utilizar dicho servicio con una elevada interoperabilidad. Es posible desacoplar la implementación del servicio web y su uso por parte de un cliente. También es posible desacoplar la implementación del servicio y de cliente. Las implementaciones concretas del servicio pueden desacoplarse a nivel de lógica y transporte. La siguiente figura muestra el diagrama de una arquitectura orientada a SOA:



Las **componentes** de los servicios web son:

- **Servicio**: la aplicación es ofrecida para ser utilizada por solicitantes que respetan los requisitos especificados por el proveedor de servicios. La implementación se realiza sobre una plataforma accesible en la red. El servicio se describe a través de un leguaje de descripción de servicio. Tanto la descripción como las políticas de uso han sido publicadas de antemano en un registro.

- **Proveedor de Servicio**: desde el punto de vista comercial, es quien presta el servicio. Desde el punto de vista de arquitectura, es la plataforma que provee el servicio.

- **Registro de Servicios**: es un depósito de descripciones de servicios que puede ser consultado, donde los proveedores de servicios publican sus servicios y los solicitantes encuentran los servicios y detalles para utilizar dichos servicios.

- **Solicitante de servicios**: desde el punto de vista comercial, la empresa que requiere cierto servicio. Desde el punto de vista de la arquitectura, la aplicación o cliente que busca e invoca un servicio.

Los servicios web son aplicaciones que pueden ser:

- **Descritos** mediante un lenguaje de descripción de servicio, como el lenguaje **WSDL** (Web Service Description Language).

- **Publicados** al someter las descripciones y políticas de uso en algún Registro bien conocido, utilizando el servicio de registro **UDDI** (Universal Description, Discovery and Integration).

- **Descubiertos** al enviar peticiones al Registro y recibir detalles del servicio que se ajusta a los parámetros de la búsqueda.

- **Asociados** (binding) al utilizar la información contenida en la descripción del servicio para crear una instancia de servicio disponible.

- **Invocados** sobre la red mediante **SOAP** al utilizar la información contenida en la descripción del servicio.

- **Transportados** a través de un protocolo de transporte (HTTP, SMTP u otros).

2. Tecnologías

El conjunto de estándares ligados a los servicios web son:

Estándar	Compañía	Descripción
XML	W3C	Lenguaje de marcado
HTTP	W3C	Protocolo de transporte
SOAP	W3C	Formato de los documentos
WSDL	W3C	Definición de los servicios
UDDI	UDDI	Servicio de registro y descubrimiento
WSIL	IBM y Microsoft	Servicio de descubrimiento

- **XML (eXtensible Markup Language)**: este estándar ha revolucionado la forma en que estructuramos, describimos e intercambiamos información. Todas las tecnologías de WS se basan en XML. El diseño de XML se deriva de 2 fuentes principales: SGML (Standard Generalized Markup Language) y de HTML (HyperText Markup Language).

- **HTTP (HyperText Transfer Protocol)**: protocolo de comunicación para la transmisión de los mensajes entre aplicaciones. Es el más habitual, no el único.

- **SOAP (Simple Object Access Protocol)**: es el protocolo de comunicación, sobre la capa de transporte basado en XML, que sirve para invocar los servicios a través de un protocolo siendo los más habituales HTTP o SMTP, pero realmente es independiente y permite otros cómo POP3 o JMS. Permite tanto describir el contenido del mensaje y reglas de codificación de los tipos de datos, como aspectos de seguridad y transaccionalidad. Se encuentra estandarizado por el W3C, lo que garantiza la comunicación entre sistemas heterogéneos que lo implementen.

- **WSDL (Web Service Description Language)**: es el estándar propuesto para la descripción de los servicios Web, el cual consiste en un lenguaje basado en XML y XML Schema, que define la interfaz de servicio y sus características de implementación. El WSDL es apuntado en los registros UDDI y describe los mensajes SOAP que definen un servicio Web en particular.

- **UDDI (Universal Description, Discovery and Integration)**: es un estándar para describir las interfaces técnicas que permitan el acceso a servicios web. Mantiene un directorio donde se publican los servicios proporcionando la información necesaria para permitir su invocación. Presenta dos API que permiten a los servicios publicar sus funcionalidades y a los clientes poder descubrir los WS mediante el envío de peticiones. Cada servicio se publica en el UDDI proporcionando la URL de su WSDL y metainformación.

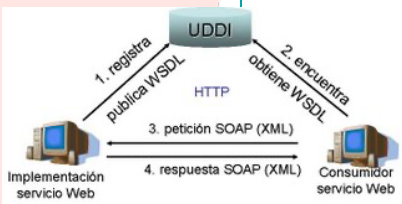
  De manera general se entiende que UDDI proporciona tres tipos de servicios:

  - **Páginas blancas**: información general sobre los proveedores de los servicios.
  - **Páginas amarillas**: categorías y clasificaciones de servicios.
  - **Páginas verdes**: las reglas de negocio o información técnica sobre los servicios.

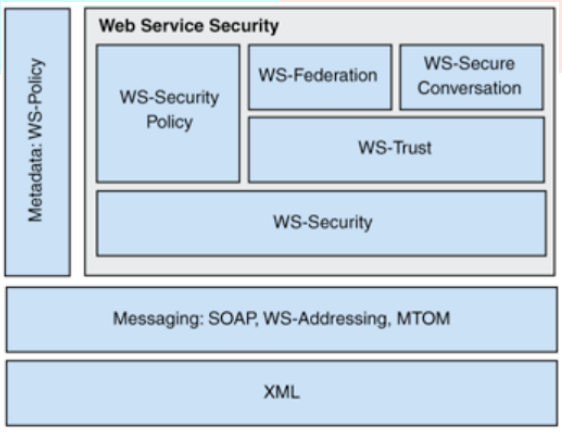
- **WSIL (Web Services Inspection Language)**: es un método alternativo a UDDI para el descubrimiento de servicios, que puede ser complementario a UDDI. WSIL es un enfoque alternativo para el descubrimiento de WS que permite ir directamente al proveedor de servicios y preguntar por los servicios que proporciona. WSIL proporciona un modelo basado en XML para construir un conjunto de referencias a descripciones de WS.

OPERATIVA

Así, WSDL se usa a menudo en combinación con SOAP y XML Schema. Un programa cliente que se conecta a un servicio web puede leer el WSDL para determinar que funciones están disponibles en el servidor. Los tipos de datos especiales se incluyen en el archivo WSDL en forma de XML Schema. El cliente puede usar SOAP para hacer la llamada a una de las funciones listadas en el WSDL.



En relación con la seguridad de los servicios web, tenemos el **Web Service Security** (o WS-Security). Se muestra el esquema de la pila de especificaciones:



- **WS-Security**: estándar de OASIS como extensión de SOAP, que provee mecanismos para reforzar la integridad y confidencialidad de los mensajes, y la autenticación de un mensaje individual. Especifica un procedimiento que permite la comunicación de tokens de seguridad (son declaraciones de seguridad añadidas a los mensajes) como SAML o Kerberos.

  Respecto a la autenticación, utiliza un **token de seguridad** para validar al usuario y determinar si un cliente es válido. Para ello, se inserta un token de seguridad en el mensaje de solicitud SOAP. Se admiten los siguientes tipos de token de seguridad, que representan la identidad del usuario, para la autenticación:

  - **User Name Token**: el token **UsernameToken** contiene el subelemento Username con el atributo Id, pudiendo ampliarse los atributos.
  - **Binary Security Tokens**: el token **BinarySecurityToken**, formateado en binario, contendrá un certificado X.509 o un ticket de Kerberos.
  - **XML Tokens**: permiten especificar la identidad en formato XML.
  - **EncryptedData** Token: se utiliza para encriptar tokens de seguridad.

- **WS-Secure Conversation**: creado por IBM, que trabaja junto con WS-Security, WS-Trust y WS-Policy para la creación y compartición de contenidos.

- **WS-Trust**: estándar de OASIS para trabajar con tokens de seguridad. Se incluye el proceso de solicitud, emisión y control sobre tokens de seguridad y se permite la gestión de las relaciones de confianza entre los servicios.

- **WS-Federation**: especificación de federación de identidades que describe la forma de llevar a cabo la intermediación entre los participantes. Tiene como objetivo principal ayudar a la definición de los mecanismos de federación de dominios de seguridad, ya sean distintos o similares (por ejemplo, una de las partes podrá utilizar Kerberos y la otra certificados X.509v3). Para ello, define, categoriza e intermedia con los niveles de confianza de las identidades, atributos y autenticación de los WS de todos los colaboradores.

- **WS-Policy**: estándar de OASIS encargado de delimitar las diferentes políticas aplicables a los servicios Web.

- **WS-Security Policy**: estándar de OASIS que define un framework para permitir que los servicios web expresen sus restricciones y requisitos como un conjunto de aserciones de políticas de seguridad para su uso con WS-Policy.

- **WS-Addressing**: especificación del W3C que permite la interoperabilidad entre los WS definiendo un modo estándar para direccionar los WS y proporcionar información de direccionamiento en los mensajes. Esta especificación define 2 tipos de elementos que se incorporan en el mensajes SOAP: (1) Endpoint References (EPR) son referencias de invocación que identifican el punto final al que deben ser dirigidas las peticiones; (2)

Message Information Headers son cabeceras específicas que contienen información relacionada con la identificación que caracteriza al mensaje.

- **WS-SecureConversation**: estándar de OASIS que define los mecanismos para establecer y compartir contextos de seguridad, y para obtener claves de contextos de seguridad.

También disponemos de los siguientes elementos de seguridad:

- **XML Digital Signature**: establecido por el W3C, realiza la firma digital que garantiza la integridad de las partes en una comunicación. Asimismo, proporciona autenticación de mensajes, integridad de datos, soporte de transacciones sin repudio y firmas para cualquier contenido digital o XML. En el documento se añade un elemento `Signature` que encapsula el contenido de la firma digital incluyendo una referencia al objeto firmado, la indicación del algoritmo de canonización, y el valor resultante de la firma.

```
<Signature ID?>
  <SignedInfo>
    <CanonicalizationMethod />
    <SignatureMethod />
   (<Reference URI? >
     (<Transforms>)?
      <DigestMethod>
      <DigestValue>
    </Reference>)+
  </SignedInfo>
  <SignatureValue>
 (<KeyInfo>)?
 (<Object ID?>)*
</Signature>
```

- **XML Encryption**: la encriptación XML es una recomendación del W3C que especifica el proceso para cifrar datos o documentos completos y representar esa información encriptada en un documento XML, gracias al elemento `EncryptedData`. Soporta los algoritmos TripleDES, AES y RSA.

```
<EncryptedData Id? Type? MimeType? Encoding?>
  <EncryptionMethod/>?
  <ds:KeyInfo>
    <EncryptedKey>?
    <AgreementMethod>?
    <ds:KeyName>?
    <ds:RetrievalMethod>?
    <ds:*>?
  </ds:KeyInfo>?
  <CipherData>
    <CipherValue> | <CipherReference URI?>
  </CipherData>
  <EncryptionProperties>?
</EncryptedData>
```

- **XML Key Management**: protocolo desarrollado por el W3C que está orientado a la obtención de información acerca de claves y certificados. Permite manejar los procesos de registro y revocación del servicio. Mediante el uso de este protocolo se puede intercambiar y registrar claves públicas. Está compuesto de 2 elementos fundamentales: la información (X-KISS) y el registro (X-RKSS) de la clave pública.

- SAML 2.0 **(Security Assertion Markup Language)**: estándar de OASIS que define un esquema XML para el intercambio de datos de autenticación, permitiendo el inicio de sesión único (SSO). SAML utiliza XML para representar los datos de identidad del usuario y HTTP simple para los mecanismos de transporte de datos.

SAML 2.0 se puede utilizar junto con WS-Security. La especificación SAML define 3 roles:

- **Principal**: representa al usuario que desea realizar la autenticación en una aplicación y que, por tanto, solicita un servicio.
- **Proveedor de identidad** (por ejemplo, Autentica).
- **Proveedor de servicio**: representa a la aplicación destino a la cual se desea acceder por parte del usuario.

En este escenario, el rol principal sería el usuario que desea autenticarse en una aplicación. Éste solicita acceso al proveedor de servicio, que será la aplicación a la cual desea acceder. El proveedor de servicio, a su vez, solicita y obtiene en caso de éxito, una confirmación de identidad desde el proveedor de identidad. Teniendo como base la confirmación recibida por parte del proveedor de identidad, el proveedor de servicio suministrará la respuesta que considere oportuno al usuario, permitiendo o no el acceso a la aplicación.

3. SOAP (invocación)

SOAP versión 1.2 (http://www.w3.org/TR/soap12/) es un protocolo ligero destinado a intercambiar información estructurada en un entorno descentralizado y distribuido. Utiliza tecnologías XML para definir un marco de mensajes extensible que proporciona una construcción de mensaje que se puede intercambiar a través de una variedad de protocolos subyacentes. El marco ha sido diseñado para ser independiente de cualquier modelo de programación en particular y otras semánticas de implementación específica.

Las características fundamentales son:

- **Extensibilidad**: se puede implementar seguridad (WS-Security) y enrutamiento (antes WS-Routing, ahora WS-Addressing).
- **Neutralidad**: puede ser usado sobre cualquier protocolo de transporte.
- **Independencia**: puede ser programado en cualquier lenguaje.

SOAP consta de 3 partes:

- Un **sobre** que define una estructura que describe lo que hay en un mensaje y cómo procesarlo.
- Un conjunto de **normas** para expresar instancias de tipos de datos definidos por la aplicación.
- Un conjunto de **reglas** para representar llamadas y respuestas de procedimientos remotos.

Estructura de un mensaje SOAP

Un mensaje SOAP se codifica como un documento XML, que consta de un elemento `<Envelope>`, que contiene un elemento `<Header>` opcional y un elemento `<Body>` obligatorio. El elemento `<Fault>`, que se encuentra en `<Body>`, se utiliza para notificar errores.

```
POST /example HTTP/2
Host: example.org
Content-Type: text/xml; charset=utf-8
...
<?xml version="1.0"?>
    <SOAP-ENV:Envelope
      xmlns:SOAP-
      ENV="http://www.w3.org/2003/05/soap-envelope"
      SOAP-ENV:encodingStyle
      ="http://www.w3.org/2001/12/soap-encoding">
            ...
      <SOAP-ENV:Header>
            ...
      </SOAP-ENV:Header>
      <SOAP-ENV:Body>
            ...
      </SOAP-ENV:Body>
   </SOAP_ENV:Envelope>
```



- **El SOBRE SOAP (envelope)**: describe el mensaje, a quién va dirigido y cómo debe ser procesado. El sobre incluye las definiciones de tipos que se usarán en el documento. Contiene una cabecera de forma opcional, y el cuerpo del mensaje.

  `<Envelope>` es el elemento raíz en cada mensaje SOAP y contiene dos elementos hijo, un elemento `<Header>` opcional y un elemento `<Body>` obligatorio.

- **La CABECERA SOAP (header)**: `<Header>` es un subelemento opcional del sobre SOAP y se utiliza para pasar información relacionada con la aplicación que los nodos SOAP van a procesar a lo largo de la vía de acceso del mensaje.

  Los subelementos del header se denominan **bloques de cabecera**. Los atributos definidos por SOAP incluyen:

encodingStyle	Indica las normas utilizadas para codificar las partes de un mensaje SOAP.
role	Especifica si un nodo concreto operará en un mensaje. Si el rol especificado para el nodo coincide con el atributo role del bloque de cabecera, el nodo procesa la cabecera. Si los roles no coinciden, el nodo no procesa el bloque de cabecera. En SOAP 1.1, el atributo actor realiza la misma función. Tenemos tres roles estándares además de los definidos por la aplicación: - http://www.w3.org/2003/05/soap-envelope/none: ninguno de los nodos SOAP debe procesar directamente el bloque de cabecera. - http://www.w3.org/2003/05/soap-envelope/next: todos los nodos SOAP examinan el bloque de cabecera. - http://www.w3.org/2003/05/soap-envelope/ultimateReceiver: solo el nodo final examina el bloque de cabecera.
mustUnderstand	Este atributo se utiliza para asegurar que los nodos SOAP no ignoran los bloques de cabecera que son importantes para la finalidad global de la aplicación.
relay	Cuando un nodo intermediario SOAP procesa un bloque de cabecera, el nodo intermediario SOAP elimina el bloque de cabecera del mensaje SOAP.

- **El CUERPO SOAP (body)**: contiene el mensaje en sí. `<Body>` es un subelemento obligatorio del sobre SOAP, que contiene información dirigida al destinatario final del mensaje.

- **El ERROR SOAP (fault)**: `<Fault>` es un subelemento opcional del cuerpo SOAP, que se utiliza para notificar errores al remitente de la petición.

Los elementos XML de `<Header>` y `<Body>` están definidos por las aplicaciones que hacen uso de ellos, aunque la especificación SOAP establece algunas restricciones en su estructura.

Ejemplo: mensaje SOAP aplicación reserva de viajes.

```
<?xml version="1.0" ?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
 <env:Header>
  <m:reservation xmlns:m="http://travelcompany.example.org/reservation"
          env:role="http://www.w3.org/2003/05/soap-envelope/role/next"
```

```
            env:mustUnderstand="true">
    <m:reference>uuid:093a2da1-q345-739r-ba5d-pqff98fe8j7d</m:reference>
    <m:dateAndTime>2001-11-29T13:20:00.000-05:00</m:dateAndTime>
   </m:reservation>
   <n:passenger xmlns:n="http://mycompany.example.com/employees"
           env:role="http://www.w3.org/2003/05/soap-envelope/role/next"
            env:mustUnderstand="true">
    <n:name>Åke Jógvan Øyvind</n:name>
   </n:passenger>
  </env:Header>
  <env:Body>
   <p:itinerary
     xmlns:p="http://travelcompany.example.org/reservation/travel">
    <p:departure>
      <p:departing>New York</p:departing>
      <p:arriving>Los Angeles</p:arriving>
      <p:departureDate>2001-12-14</p:departureDate>
      <p:departureTime>late afternoon</p:departureTime>
      <p:seatPreference>aisle</p:seatPreference>
    </p:departure>
    <p:return>
      <p:departing>Los Angeles</p:departing>
      <p:arriving>New York</p:arriving>
      <p:departureDate>2001-12-20</p:departureDate>
      <p:departureTime>mid-morning</p:departureTime>
      <p:seatPreference/>
    </p:return>
   </p:itinerary>
   <q:lodging
    xmlns:q="http://travelcompany.example.org/reservation/hotels">
    <q:preference>none</q:preference>
   </q:lodging>
  </env:Body>
 </env:Envelope>
```

Los elementos contenidos en el interior del header y body están definidos por la aplicación en su propio XML Schema. La cabecera contiene 2 bloques (reservation y passenger) con información de control, fecha y hora de la reserva y referencia, así como el nombre del pasajero. El cuerpo del mensaje contiene los datos del itinerario en el elemento itinerary, que contiene la información de ida y vuelta del viaje.

La respuesta que envía el servidor consiste en una aclaración para que el cliente indique cuál de los aeropuertos de New York se va a utilizar, ofreciendo 3 opciones.

Mensaje SOAP de respuesta:

```
<?xml version="1.0" ?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
 <env:Header>
  <m:reservation xmlns:m="http://travelcompany.example.org/reservation"
      env:role="http://www.w3.org/2003/05/soap-envelope/role/next"
           env:mustUnderstand="true">
   <m:reference>uuid:093a2da1-q345-739r-ba5d-pqff98fe8j7d</m:reference>
   <m:dateAndTime>2001-11-29T13:35:00.000-05:00</m:dateAndTime>
  </m:reservation>
  <n:passenger xmlns:n="http://mycompany.example.com/employees"
      env:role="http://www.w3.org/2003/05/soap-envelope/role/next"
```

```
            env:mustUnderstand="true">
     <n:name>Åke Jógvan Øyvind</n:name>
    </n:passenger>
   </env:Header>
   <env:Body>
    <p:itineraryClarification
      xmlns:p="http://travelcompany.example.org/reservation/travel">
     <p:departure>
       <p:departing>
         <p:airportChoices>
            JFK LGA EWR
         </p:airportChoices>
       </p:departing>
     </p:departure>
     <p:return>
       <p:arriving>
         <p:airportChoices>
           JFK LGA EWR
         </p:airportChoices>
       </p:arriving>
     </p:return>
    </p:itineraryClarification>
   </env:Body>
  </env:Envelope>
```

Hemos visto como los mensajes SOAP nos sirven para intercambiar cualquier documento XML entre aplicaciones. Pero puede ocurrir que necesitemos enviar en el mensaje datos que no son XML, como puede ser una imagen.

El anexo (**Attachment**) opcional, puede contener cualquier tipo de contenido (incluido el XML). De esta forma podremos enviar cualquier tipo de contenido junto a un mensaje SOAP, sea o no XML. Además, si es necesario, nuestro mensaje podrá tener tantos anexos como se necesiten

MTOM (Message Transmission Optimization Mechanism) proporciona una forma más efectiva y normalizada por W3C de transmitir ficheros, donde en lugar de proporcionar el contenido en el body codificado en Base64, envía los datos binarios como adjunto.

A efectos prácticos es recomendable la utilización del método de transmisión MTOM siempre que se deban transmitir adjuntos, salvo cuando el tamaño máximo de los ficheros objeto de la transmisión no superen en ningún caso el tamaño de 1MB.

MTOM es normalmente usado con **XOP** (XML-binary Optimized Packaging).

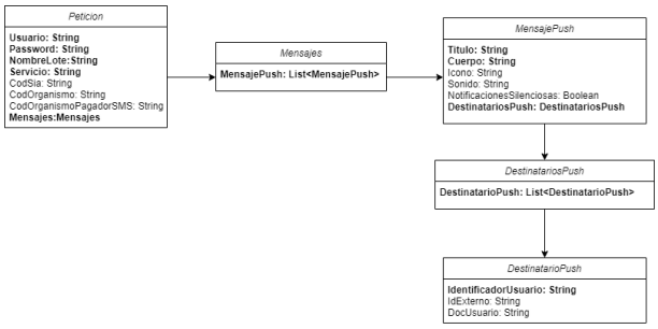
Ejemplo: consulta temperatura.

```
<SOAP-ENV:Envelope
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
    <SOAP-ENV:Body>
        <ns:getTemperatura xmlns:ns="http://localhost">
            <area>Granada</area>
        </ns:getTemperatura>
    </SOAP-ENV:Body>
```

```
</SOAP-ENV:Envelope>
```

Llamada al método getTemperatura para obtener información meteorológica, proporcionando como parámetro el área de la que se quiere obtener la temperatura.

Ejemplo: envío de un aviso PUSH a través de la plataforma SIM y respuesta obtenida.



Petición XML:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:pet="http://misim.redsara.es/misim-bus-webapp/peticion">
  <soapenv:Header/>
  <soapenv:Body>
    <pet:Peticion>
      <pet:Usuario>appMOVILpush</pet:Usuario>
      <pet:Password>appMOVILpush</pet:Password>
      <pet:NombreLote>662_1230</pet:NombreLote>
      <pet:Servicio>662</pet:Servicio>
      <pet:Mensajes>
        <pet:MensajePush>
          <pet:Titulo>Titulo 2</pet:Titulo>
          <pet:Cuerpo>{"idServicioMovil":"2","title":"Newsletter del Portal de Administración
Electrónica", "bodyMessage":"Newsletter consulta tus notificaciones YA!", "url":
"http://google.com", "icono":""}</pet:Cuerpo>
          <pet:DestinatariosPush>
            <pet:DestinatarioPush>
              <pet:IdentificadorUsuario>662_1230</pet:IdentificadorUsuario>
            </pet:DestinatarioPush>
          </pet:DestinatariosPush>
        </pet:MensajePush>
      </pet:Mensajes>
    </pet:Peticion>
  </soapenv:Body>
</soapenv:Envelope>
```

Respuesta para la petición anterior:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <Respuesta xmlns="http://misim.redsara.es/misim-bus-webapp/respuesta">
      <Status>
        <StatusCode>1000</StatusCode>
        <StatusText>OK</StatusText>
        <Details>Peticion procesada correctamente</Details>
      </Status>
      <Lote>
        <idLote>48642</idLote>
      </Lote>
      <Mensajes>
        <Mensaje>
          <idExterno/>
          <idMensaje>60872</idMensaje>
        </Mensaje>
      </Mensajes>
    </Respuesta>
  </soapenv:Body>
</soapenv:Envelope>
```

4. WSDL (descripción)

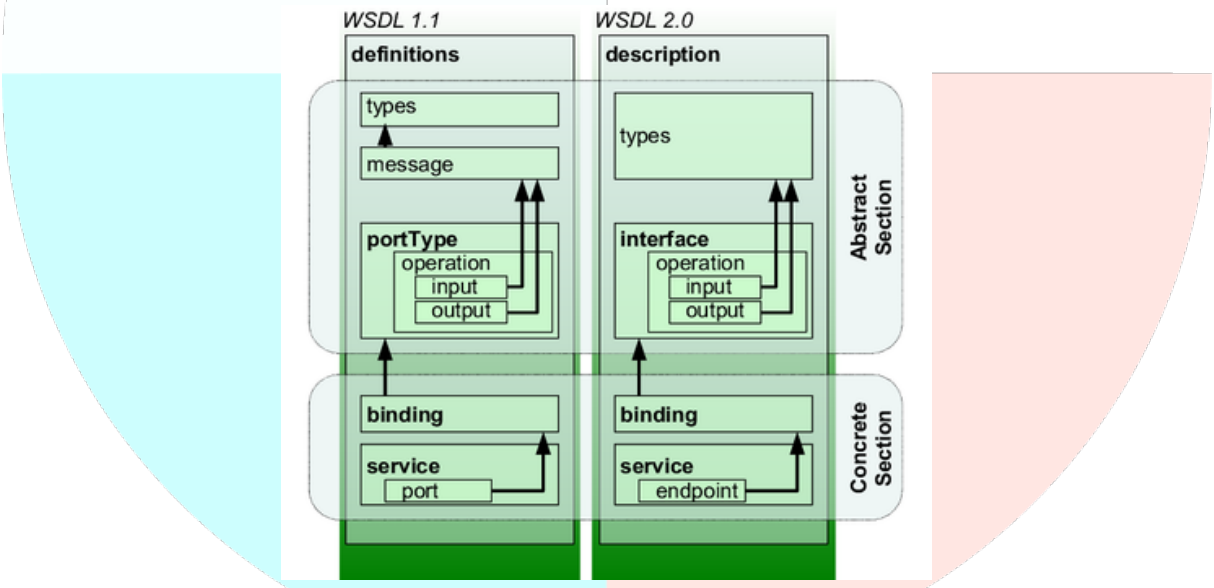
El lenguaje de descripción de servicios web versión 2.0 (WSDL 2.0) proporciona un modelo y un formato XML para **describir servicios web**.

WSDL 2.0 permite separar la descripción de la funcionalidad abstracta ofrecida por un servicio de los detalles concretos de una descripción del servicio, como "cómo" y "dónde" se ofrece esa funcionalidad.

Esta especificación define un lenguaje para describir la funcionalidad abstracta de un servicio, así como un marco para describir los detalles concretos de una descripción de servicio.

Es un lenguaje específico XML que permite definir la **interfaz** de un servicio web. Una descripción WSDL (fichero WSDL) de un servicio web proporciona una descripción entendible por la máquina de la interfaz del servicio Web, indicando cómo se debe llamar al servicio, qué parámetros espera, y qué estructuras de datos devuelve.

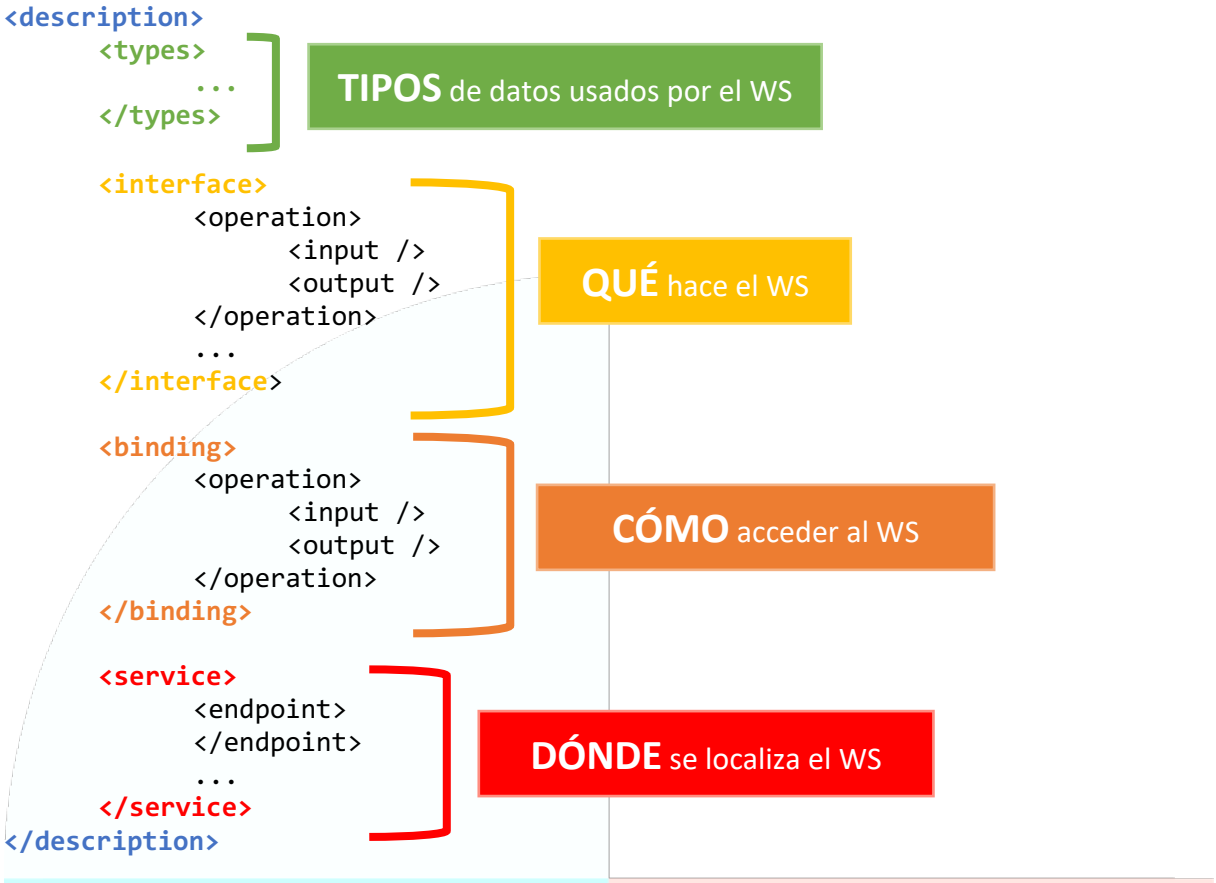
En la versión actual de WSDL se ha cambiado el significado de la **D** (**Definition**) respecto de la versión 1.1. La versión 2.0 plantea cambios de nomenclatura y estructura del fichero XML que contiene la descripción del servicio. En la figura siguiente mostramos la estructura que siguen los ficheros WSDL en ambas versiones, en donde podemos observar el cambio de nomenclatura en la versión 2.0.



Como podemos observar se definen dos secciones:

- **Sección ABSTRACTA**: se especifican las características del servicio web como el nombre de la operación o los tipos de datos de entrada y salida.
- **Sección CONCRETA**: se define el tipo de protocolo concreto de comunicación o el formato de mensaje que usa el servicio web.

La **estructura** de un documento WSDL versión 2.0 es:



WSDL está formado por los siguientes **elementos**:

- **<description>**: elemento raíz del documento WSDL.

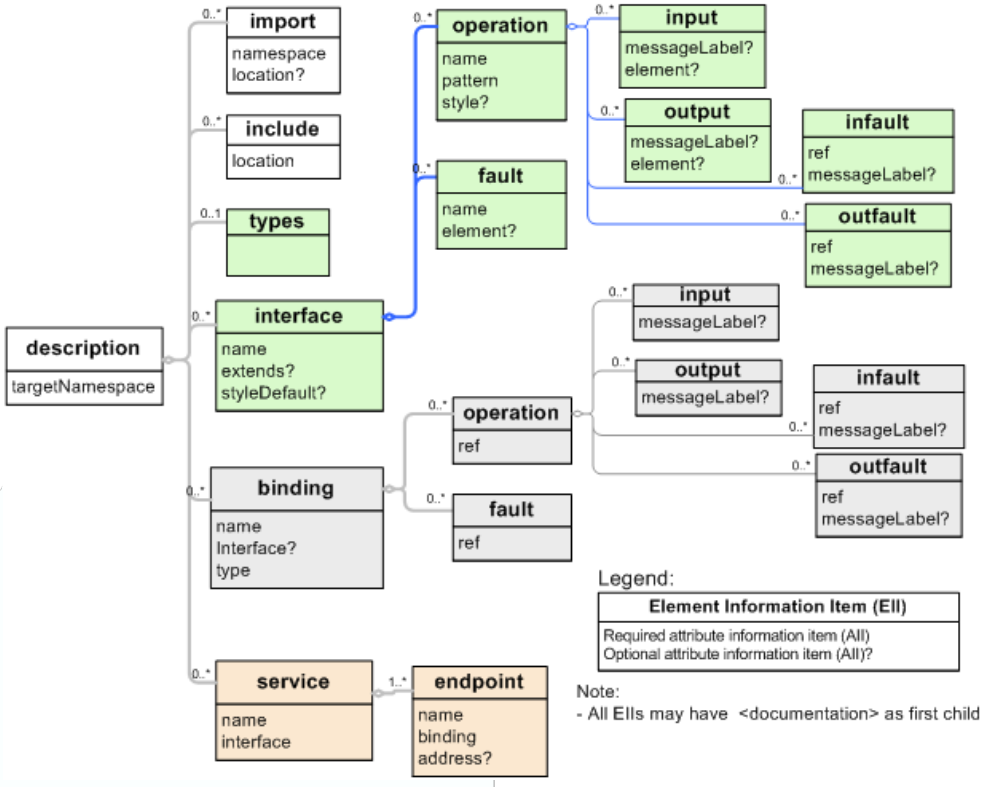
- **<types>**: define los tipos de datos usados por el WS. Se recomienda el uso de XML Schema (XSD) para ello, aunque es posible usar cualquier tipo de lenguaje. En WSDL 1.1 <messages> definía los datos (types) que están involucrados en una determinada operación.

- **<interface>** (antes, portType): describe la funcionalidad abstracta del WS, las operaciones que pueden realizar y los mensajes necesarios para llevar a término dichas operaciones.

- **<binding>**: especifica los protocolos concretos para acceder al servicio (el cómo).

- **<service>**: define el conjunto de funciones que han sido expuestas en el WS (el dónde). Con **<endpoint>** se indica la dirección para acceder a un servicio web (normalmente una dirección URI).

El siguiente diagrama ofrece una descripción general del conjunto de información XML para un documento WSDL 2.0:



WSDL define cuatro tipos básicos de operaciones, llamadas **primitivas de transmisión**. Estos estilos de operaciones representan los patrones de utilización más normales en servicios web, y surgen de las posibles combinaciones de los subelementos input y output dentro de un elemento operation de una interface de WSDL. Tenemos:

- **Operación ONE-WAY (en una dirección)**: describe un servicio en el que se recibe un mensaje por parte del cliente, pero no se envía respuesta alguna.

- **Operación REQUEST-RESPONSE (petición/respuesta)**: describe un servicio en el que se recibe un mensaje por parte del cliente y devuelve otro como respuesta al primero.

- **Operación SOLICIT-RESPONSE (solicitud/respuesta)**: describe un servicio en el que éste envía un mensaje y posteriormente recibe una respuesta.

- **Operación NOTIFICATION (Notificación)**: describe un servicio en el que éste envía un mensaje y no recibe respuesta alguna.

La tabla siguiente muestra el conjunto de espacios de nombres utilizados en WDSL, así como el prefijo típico asociado a cada uno de ellos. Puede ser necesaria la utilización de otros espacios de nombres para bindings adicionales (por ejemplo, SMTP):

Prefijo	Namespace
wsdl	http://www.w3.org/ns/wsdl
wsdli	http://www.w3.org/ns/wsdl-instance
wsdlx	http://www.w3.org/ns/wsdl-extensions
wprc	http://www.w3.org/ns/wsdl/rpc
wsoap	http://www.w3.org/ns/wsdl/soap
whttp	http://www.w3.org/ns/wsdl/http
xs	http://www.w3.org/2001/XMLSchema
xsi	http://www.w3.org/2001/XMLSchema-instance

Ejemplo: endpoint (WSDL) para envío de un aviso PUSH a través de la plataforma SIM.

```
▼<wsdl:definitions xmlns:impl="http://misim.redsara.es/misim-bus-webapp/" xmlns:ns1="http://misim.redsara.es/misim-bus-webapp/peticion"
 xmlns:ns2="http://misim.redsara.es/misim-bus-webapp/respuesta" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
 xmlns:xsd="http://www.w3.org/2001/XMLSchema" targetNamespace="http://misim.redsara.es/misim-bus-webapp/">
  ▼<wsdl:types>
    ▼<schema xmlns="http://www.w3.org/2001/XMLSchema" xmlns:impl="http://misim.redsara.es/misim-bus-webapp/" xmlns:ns1="http://misim.redsara.es/misim-bus-
     webapp/peticion" xmlns:ns2="http://misim.redsara.es/misim-bus-webapp/respuesta" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
     xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" targetNamespace="http://www.map.es/xml-schemas">
       <import namespace="http://misim.redsara.es/misim-bus-webapp/peticion" schemaLocation="https://misim.redsara.es/misim-bus-webapp/EnvioMensajesService?
       xsd=peticion.xsd"/>
       <import namespace="http://misim.redsara.es/misim-bus-webapp/respuesta" schemaLocation="https://misim.redsara.es/misim-bus-webapp/EnvioMensajesService?
       xsd=respuesta.xsd"/>
     </schema>
  </wsdl:types>
  ▼<wsdl:message name="enviarMensajePeticion">
     <wsdl:part element="ns1:Peticion" name="peticion"> </wsdl:part>
   </wsdl:message>
  ▼<wsdl:message name="enviarMensajeRespuesta">
     <wsdl:part element="ns2:Respuesta" name="respuesta"> </wsdl:part>
   </wsdl:message>
  ▼<wsdl:portType name="EnvioMensajesServiceWSBindingPortType">
    ▼<wsdl:operation name="enviarMensaje">
       <wsdl:input message="impl:enviarMensajePeticion" name="Peticion"> </wsdl:input>
       <wsdl:output message="impl:enviarMensajeRespuesta" name="Respuesta"> </wsdl:output>
     </wsdl:operation>
   </wsdl:portType>
  ▼<wsdl:binding name="EnvioMensajesServiceBinding" type="impl:EnvioMensajesServiceWSBindingPortType">
     <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    ▼<wsdl:operation name="enviarMensaje">
       <soap:operation soapAction="enviarMensaje" style="document"/>
      ▼<wsdl:input>
         <soap:body parts="peticion" use="literal"/>
       </wsdl:input>
      ▼<wsdl:output>
         <soap:body parts="respuesta" use="literal"/>
       </wsdl:output>
     </wsdl:operation>
   </wsdl:binding>
  ▼<wsdl:service name="EnvioMensajesService">
    ▼<wsdl:port binding="impl:EnvioMensajesServiceBinding" name="EnvioMensajesServicePort">
       <soap:address location="https://misim.redsara.es/misim-bus-webapp/EnvioMensajesService"/>
     </wsdl:port>
   </wsdl:service>
</wsdl:definitions>
```

Ejemplo: endpoint (WSDL) para consulta en REA.

```
▼<wsdl:definitions xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
 xmlns:ns2="https://consultaApoderamientos.minhap.es/respuestaConsulta" xmlns:ns1="https://consultaApoderamientos.minhap.es/peticionConsulta"
 xmlns:ns0="https://consultaApoderamientos.minhap.es/soapfault" xmlns:impl="https://consultaApoderamientos.minhap.es/"
 targetNamespace="https://consultaApoderamientos.minhap.es/">
  ▼<wsdl:types>
    ▼<schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
     xmlns:ns2="https://consultaApoderamientos.minhap.es/respuestaConsulta" xmlns:ns1="https://consultaApoderamientos.minhap.es/peticionConsulta"
     xmlns:ns0="https://consultaApoderamientos.minhap.es/soapfault" xmlns:impl="https://consultaApoderamientos.minhap.es/" xmlns="http://www.w3.org/2001/XMLSchema"
     targetNamespace="http://www.map.es/xml-schemas">
       <import namespace="https://consultaApoderamientos.minhap.es/peticionConsulta" schemaLocation="https://reaws.redsara.es/reaCXFServer/services/reaCXFWSv2?
       xsd=peticionConsulta.xsd"/>
       <import namespace="https://consultaApoderamientos.minhap.es/respuestaConsulta" schemaLocation="https://reaws.redsara.es/reaCXFServer/services/reaCXFWSv2?
       xsd=respuestaConsulta.xsd"/>
       <import namespace="https://consultaApoderamientos.minhap.es/soapfault" schemaLocation="https://reaws.redsara.es/reaCXFServer/services/reaCXFWSv2?
       xsd=soapfault.xsd"/>
     </schema>
  </wsdl:types>
 ▼<wsdl:message name="peticionConsulta">
    <wsdl:part element="ns1:PeticionConsulta" name="peticion"> </wsdl:part>
  </wsdl:message>
 ▼<wsdl:message name="soapFault">
    <wsdl:part element="ns0:NotificacionesFault" name="soapFault"> </wsdl:part>
  </wsdl:message>
 ▼<wsdl:message name="respuestaConsulta">
    <wsdl:part element="ns2:RespuestaConsulta" name="respuesta"> </wsdl:part>
  </wsdl:message>
 ▼<wsdl:portType name="ConsultaAvanzadaPortType">
  ▼<wsdl:operation name="consultaAvanzadaApoderamientos">
     <wsdl:input message="impl:peticionConsulta" name="PeticionConsulta"> </wsdl:input>
     <wsdl:output message="impl:respuestaConsulta" name="RespuesConsulta"> </wsdl:output>
     <wsdl:fault message="impl:soapFault" name="SoapFault"> </wsdl:fault>
    </wsdl:operation>
  </wsdl:portType>
 ▼<wsdl:binding name="ConsultaAvanzadaBinding" type="impl:ConsultaAvanzadaPortType">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  ▼<wsdl:operation name="consultaAvanzadaApoderamientos">
     <soap:operation soapAction="consultaAvanzadaApoderamientos" style="document"/>
    ▼<wsdl:input>
       <soap:body parts="peticion" use="literal"/>
     </wsdl:input>
    ▼<wsdl:output>
       <soap:body parts="respuesta" use="literal"/>
     </wsdl:output>
    ▼<wsdl:fault name="SoapFault">
       <soap:fault name="SoapFault" use="literal"/>
     </wsdl:fault>
    </wsdl:operation>
  </wsdl:binding>
 ▼<wsdl:service name="ConsultaAvanzadaService">
  ▼<wsdl:port binding="impl:ConsultaAvanzadaBinding" name="ConsultaAvanzadaPort">
     <soap:address location="https://reaws.redsara.es/reaCXFServer/services/reaCXFWSv2"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
```

5. Ejemplos

Podemos citar, entre otros, los siguientes ejemplos:

- Callejero Catastro
- AEAT
- DEHú
- DIR3

Como podemos observar muchos servicios web se describen con WSDL 1.1. Sin embargo, actualmente está en vigor WSDL 2.0. El W3C nos ofrece una utilidad para convertir un fichero de una versión a otra de forma automática: https://www.w3.org/2006/02/WSDLConvert.html

6. ARQUITECTURA REST

REST (**REpresentational State Transfer**) representa un modelo de comunicación donde cada petición HTTP contiene la información necesaria para responder a la petición sin tener que almacenar el estado de la sesión. En REST todos los servicios son recursos, identificados por URIs y se diseñan sus representaciones mediante XML, JSON o microformatos. REST representa una arquitectura SOA que no hace uso de Servicios web SOAP ni RPC.

REST (REpresentational State Transfer) es un **estilo arquitectónico** utilizado como modelo para diseñar sistemas en entornos distribuidos y usado normalmente para definir una interfaz de transmisión sobre HTTP de manera análoga a como lo hace SOAP. REST marca las bases necesarias para crear lo que se conoce como **servicios web ligeros**.

REST surge como un apartado dentro de la tesis de uno de los creadores de la Arquitectura de la Web, Roy Thomas Fielding, año 2000. Está muy de moda actualmente gracias a la explosión de la llamada Web 2.0, ya que este estilo arquitectónico está incluido dentro del marco de la Web 2.0 por ser la base que facilita el cumplimiento de la filosofía seguida por esta corriente. Como primer acercamiento a REST, empezamos por conocer el significado sus siglas:

- **REpresentational**: significa que el servidor devuelve una representación del recurso cuando el cliente accede a una determinada URI.
- **State**: la representación devuelta por el servidor sitúa al cliente en un estado concreto.
- **Transfer**: el cliente puede ir cambiando de estado accediendo a las URIs disponibles en forma de enlace dentro de cada representación de un recurso.

Es decir, REST se basa en acceder a recursos disponibles en la Web por medio de su identificador (URI), obteniendo como resultado una representación del mismo. Esta representación plasma el estado del recurso al que representa en un momento concreto del tiempo. Además, la representación puede contener enlaces a otros recursos de modo que el cliente puede navegar a través de ellos, y consecuentemente, cambiando de estado.

La motivación de desarrollar REST era la de crear un modelo de arquitectura que describiese como debería funcionar la Web, así como que pudiese servir como marco de trabajo para los estándares de protocolos Web. REST ha sido aplicado para describir la arquitectura Web deseada, ayudar a identificar problemas existentes, comparar soluciones alternativas, y asegurar que el protocolo no viole las restricciones que hacen que la Web funcione correctamente. Este trabajo fue realizado por el Internet Engineering Taskforce (IETF) y el World Wide Web Consortium (W3C), gracias a sus esfuerzos por definir un estándar de arquitectura para la Web: HTTP, URI y HTML.

El término REST ha ido evolucionando a lo largo del tiempo. Fielding lo concibió de una manera abstracta, se refería originalmente a un conjunto de principios de arquitectura, en la actualidad se usa en el sentido más amplio para describir cualquier interfaz Web simple que utiliza XML y HTTP, sin las abstracciones adicionales de los protocolos basados en patrones de intercambio de mensajes como el protocolo SOAP introducido anteriormente. Es posible diseñar sistemas de servicios Web de acuerdo con el estilo arquitectónico REST de Fielding, y

también es posible diseñar interfaces XML/HTTP de acuerdo con el estilo de llamada a procedimiento remoto, pero sin usar SOAP. Estos dos usos diferentes del término REST causan cierta confusión en las discusiones técnicas, aunque RPC no es un ejemplo de REST. En cualquier caso, al centrarse sólo en estas tecnologías, se puede perder en parte la esencia propuesta por Fielding.

En cualquier caso, los sistemas que siguen los principios REST, es decir, que implementan esta arquitectura, se denominan **RESTful**.

La Web ha disfrutado de escalabilidad como resultado de una serie de diseños fundamentales clave:

- Un **protocolo cliente/servidor SIN ESTADO**: cada mensaje HTTP contiene toda la información necesaria para comprender la petición. Como resultado, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre mensajes. Sin embargo, en la práctica, muchas aplicaciones basadas en HTTP utilizan cookies y otros mecanismos para mantener el estado de la sesión (algunas de estas prácticas, como la reescritura de URLs, no son permitidas por REST).

- Un **conjunto de operaciones** bien definidas que se aplican a todos los recursos de información: HTTP en sí define un conjunto pequeño de operaciones, las más importantes son **POST**, **GET**, **PUT** y **DELETE**. Con frecuencia estas operaciones se equiparan a las operaciones CRUD (Create, Retrieve, Update y Delete) que se requieren para la persistencia de datos, aunque POST no encaja exactamente en este esquema.

- Una **SINTAXIS UNIVERSAL** para identificar los recursos. En un sistema REST, cada recurso es direccionable únicamente a través de su **URI**.

- El **USO DE HIPERMEDIOS**, tanto para la información de la aplicación como para las transiciones de estado de la aplicación: la representación de este estado en un sistema REST son típicamente HTML o XML. Como resultado de esto, es posible navegar de un recurso REST a muchos otros, simplemente siguiendo enlaces sin requerir el uso de registros u otra infraestructura adicional.

A partir de los principios básicos anteriores, se definen una serie de restricciones que ha de cumplir una aplicación para poder considerarse REST, y que por tanto se pueden considerar como las características principales de REST. Esto deriva en que REST no es una tecnología en sí misma, sino una arquitectura que puede implementarse de muy diversas maneras, y que simplemente ha de cumplir una serie de restricciones:

- **Cliente-Servidor**: imponer una arquitectura cliente-servidor. La separación de roles en las entidades cliente y servidora es la base para desarrollar este tipo de arquitecturas. De esta forma, es posible desarrollar estos componentes de forma paralela e independiente. Separando la interfaz de usuario de las características de almacenamiento y tratamiento de los datos se garantiza la portabilidad de esta funcionalidad de usuario entre diferentes plataformas y mejora la escalabilidad ya que simplifica los componentes del servidor.

- **Sin estado (Stateless)**: toda información que viaja en un elemento de comunicación atómico entre cliente-servidor (petición de servicio) es autocontenida, y no se almacena ningún tipo de estado del cliente por parte del servidor. Esta restricción delega toda la responsabilidad de mantenimiento de un posible estado de la comunicación al cliente.

  Lo anterior se traduce en la independencia del servidor a la hora de procesar una petición en particular, añadiendo al modelo mejoras en relación con:

  - Transparencia. Todas las peticiones en la comunicación son, por así decirlo, iguales.

  - Fiabilidad. Ante una pérdida de servicio por parte del servidor y el posterior restablecimiento de este no hay ninguna pérdida de información, y la comunicación puede volver a ser establecida sin problemas por las entidades.

  - Escalabilidad. El servidor, debido a que procesa las peticiones de forma individual y, por tanto, no tiene la necesidad de tener en cuenta la relación con peticiones anteriores, no reserva recursos. Esto hace que el servidor no tenga que realizar una gestión de recursos adicional al procesado de la petición que le ofrece al cliente, y estos recursos son liberados más fácilmente.

  La desventaja de esta restricción es que puede empeorar el funcionamiento de la red porque incrementa el tráfico de datos repetidos al enviar una serie de peticiones. Esto ocurre porque los datos no pueden quedarse almacenados en el servidor identificando un contexto determinado de comunicación. Además, poniendo el estado de la aplicación en el lado del cliente, se reduce el control para obtener un comportamiento consistente en la aplicación. La aplicación se hace muy dependiente de una correcta implementación de la semántica en las distintas versiones del cliente.

- **Cache**: con objeto de mejorar la eficiencia de la red, las respuestas a una petición deben poder ser etiquetadas como cacheable o no cacheable. Si una respuesta es cacheable, entonces al cliente se le da permiso para reutilizar la respuesta más tarde si se hace una petición equivalente.

  La consecuencia directa de esta restricción ya se ha comentado anteriormente: se mejora la eficiencia de la red al reducir el tráfico que viaja por ella al disponerse de caché. De igual manera, el tiempo de respuesta se verá también reducido.

- **Interfaz uniforme**: las restricciones que se describen a continuación son realmente las novedades que pretende aportar REST a las nuevas arquitecturas Web. La principal característica que distingue a REST del resto de estilos de arquitecturas de red es el énfasis de usar una interfaz uniforme entre los componentes. Aplicando los principios de generalidad de la ingeniería del software a los componentes de la interfaz, se simplifica la arquitectura del sistema global y la visibilidad de interacciones se mejora. Las implementaciones se separan de los servicios que proporcionan, lo que anima al desarrollo independiente.

La desventaja de usar una interfaz uniforme es que degrada la eficiencia porque la información transferida está en una forma estandarizada y no según las necesidades que tenga la aplicación. El interfaz de REST está diseñado para ser eficiente con transferencias de datos de hipermedios (audio, video y texto, con el que pueden interactuar los usuarios), que suelen ser datos voluminosos. Con esta decisión, está optimizado para la mayor parte de la Web pero no siendo así para otras formas de arquitectura de interacción. Para obtener una interfaz uniforme, REST define 4 restricciones de interfaz:

- Identificación de recursos.
- Manipulación de recursos a través de sus representaciones.
- Mensajes autodescriptivos.
- Hipermedios como el motor del estado de la aplicación (**HATEOAS**).

Para la transferencia de datos en un sistema REST, este aplica acciones concretas (POST, GET, PUT y DELETE) sobre los recursos, siempre y cuando estén identificados con una URI. Esto facilita la existencia de una interfaz uniforme que sistematiza el proceso con la información.

- **Sistema de capas**. Para poder mejorar el comportamiento de la escalabilidad en Internet, se añade la restricción del sistema de capas. Este sistema permite tener una arquitectura compuesta por capas jerárquicas, limitando el comportamiento de los componentes porque no pueden acceder más allá de la capa con la que están interactuando.

Restringiendo la visibilidad del sistema a una sola capa se limita la complejidad total de la misma y se promueve el desarrollo de las capas inferiores, siendo dicho desarrollo independiente.

Las características anteriores se manifiestan en:

- Abstracción de servicios. Delegación de servicios en capas inferiores.
- Protección de los nuevos servicios. Simplificando los componentes al mover la funcionalidad atípica de un usuario a una capa intermedia. Estas capas intermedias pueden usarse para mejorar la escalabilidad del sistema permitiendo el balanceo de servicios.

Esta restricción posee también inconvenientes, la principal desventaja es que se introduce un tiempo extra de procesamiento de datos entre capas, lo que podría ocasionar una degradación en la interactividad que percibe el cliente. Para el caso de un sistema basado en caché que utilice capas intermedias, este efecto indeseado de degradación de servicio se puede reducir incorporando el sistema de caché explicado anteriormente en las capas intermedias.

Las capas situadas en los límites de la organización deben implementar políticas de seguridad para garantizar el correcto intercambio de datos con entidades externas al dominio de dicha organización. Esto puede ser requerido para implementar el correcto funcionamiento de firewalls, haciendo coincidir las políticas de restricción del cortafuegos

con las políticas de seguridad implementadas en las capas situadas en los límites de la organización.

- **Código bajo demanda**: esta restricción es opcional y consiste en permitir a los clientes tener la funcionalidad de descargar y ejecutar código en forma de applets y scripts. Esto simplifica el lado del cliente porque reduce el número de funcionalidades que debe tener implementadas al crearse. Las funcionalidades se pueden descargar posteriormente aumentando así la extensibilidad del sistema.

  La principal desventaja de esta restricción es que reduce la visibilidad y puede influir en la seguridad del sistema. Sin embargo, tiene un propósito en el diseño arquitectónico de un sistema que abarque límites de organización múltiples. Con esta restricción, la arquitectura gana solamente una ventaja y sufre el resto de las desventajas, por ello, al ser una restricción opcional, en los contextos en los que el código bajo demanda no sea útil ni necesario, lo mejor será no incluirlo porque puede acarrear más problemas que beneficios.

Ejemplo REST:

Recurso	POST	GET	PUT	DELETE
/libros	Crear un nuevo libro	Recuperar un nuevo libro	Actualización masiva de libros	Eliminar todos los libros
/libros/1	Error	Recuperar los detalles del libro 1	Actualizar los detalles del libro 1 si existe	Eliminar el libro 1
/libros/1/autor	Crear un nuevo autor para el libro 1	Recuperar el autor del libro 1	Actualizar el autor del libro 1	Eliminar el autor del libro 1

1. Servicios web RESTful

El desarrollo de servicios web REST es similar al desarrollo de aplicaciones web. Sin embargo, la diferencia fundamental entre el desarrollo de aplicaciones web tradicionales y las más modernas es cómo pensamos sobre las acciones a realizar sobre nuestras abstracciones de datos.

De forma más concreta, el desarrollo moderno está centrado en el concepto de **nombres** (intercambio de recursos). El desarrollo tradicional está centrado en el concepto de verbos (acciones remotas a realizar sobre los datos). Con la primera forma, se está implementando un servicio web RESTful. Con la segunda, un servicio similar a una llamada a procedimiento remoto (RPC). Y, lo más importante, un servicio RESTful modifica el estado de los datos a través de la representación de los recursos (por el contrario, una llamada a un servicio RPC, oculta la representación de los datos y en su lugar envía comandos para modificar el estado de los datos en el lado del servidor). Finalmente, en el desarrollo moderno de aplicaciones web se limitan la ambigüedad en el diseño y la implementación debido a que se tienen 4

acciones específicas a realizar sobre los recursos: *Create, Retrieve, Update, Delete (CRUD)*. Por otro lado, en el desarrollo tradicional de aplicaciones web, es posible disponer de otras acciones con nombres o implementaciones no estándar.

Básicamente, el funcionamiento de en un servicio web RESTful lo podemos resumir en:

- Una petición HTTP con la URL que representa el **recurso**.

```
http://www.tai.es/tema/14
```

- Un método HTTP (HTTP Verbs) que representa la **operación**.

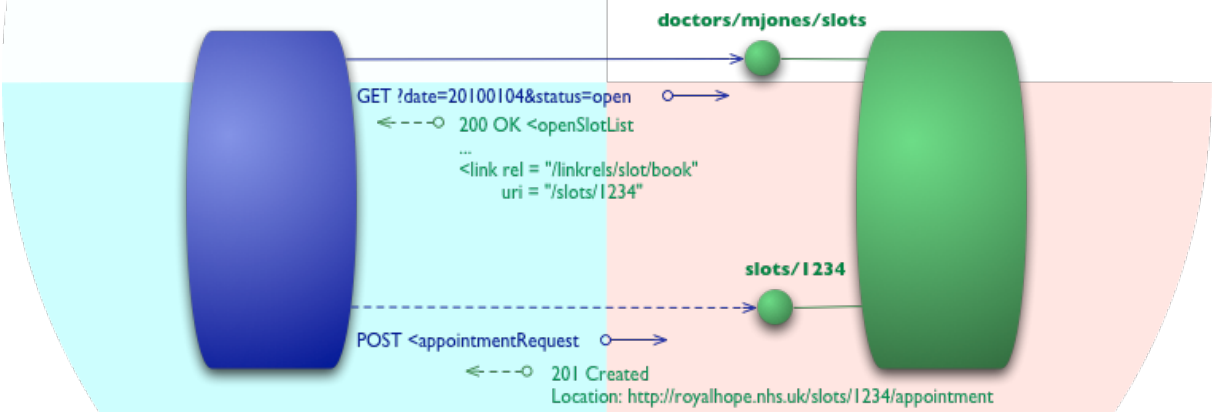
```
GET http://www.tai.es/tema/14
```

- El **formato de intercambio** de los datos es XML o JSON.

```
<alumno>
   <nombre>Juan</nombre>
   <edad>30</edad>
   <link>/alumnos/Juan</link>
</alumno>
```

- Un código de estado HTTP que representa el **resultado**.

```
200 OK HTTP/2
```



A continuación, mostramos la correspondencia entre las acciones CRUD sobre los datos y los métodos HTTP correspondientes:

Acción sobre los datos	Protocolo HTTP equivalente
CREAR nuevo recurso	**POST**
RECUPERAR datos	**GET**
ACTUALIZAR datos	**PUT**
BORRAR el recurso	**DELETE**

En su forma más simple, los servicios web RESTful son aplicaciones cliente-servidor a través de la red que manipulan el estado de los recursos. En este contexto, la manipulación de los recursos significa creación de recursos, recuperación, modificación y borrado. Sin embargo, los servicios web RESTful no están limitados solamente a estos cuatro conceptos básicos de manipulación de datos. Por el contrario, los servicios RESTful pueden ejecutar lógica en el lado del servidor, pero recordando que cada respuesta debe ser una representación del recurso del dominio en cuestión. Deberemos determinar qué operación HTTP se ajusta mejor a la manipulación que deseamos realizar sobre los datos. Mención especial merece el método PUT, ya que no se trata simplemente de una actualización de los datos, sino de establecer el estado del recurso, exista previamente o no.

Una interfaz uniforme centra la atención en los conceptos abstractos vistos: **recursos**, **representaciones** y **URIs**. Por lo tanto, si se consideran todos estos conceptos en su conjunto, se puede describir el desarrollo RESTful de las siguientes formas:

- Se utilizan URIs para conectar clientes y servidores para intercambiar recursos en forma de sus representaciones.

- En una arquitectura con estilo REST, los clientes y servidores intercambian representaciones de los recursos utilizando un protocolo e interfaces estandarizados.

Tipos de peticiones HTTP

A continuación, vamos a ver los cuatro tipos de peticiones HTTP con detalle, y veremos cómo se utiliza cada una de ellas para intercambiar representaciones para modificar el estado de los recursos.

(1) GET

El método GET se utiliza para **RECUPERAR** datos. Antes de indicar la mecánica de la petición GET, vamos a determinar cuál es el recurso que vamos a manejar y el tipo de representación que vamos a utilizar. Para ello se seguirá un ejemplo de un servicio web que gestiona alumnos en una clase, con la URI: http://restfultai.com/. Para dicho servicio, se asume una representación como la siguiente:

```
<alumno>
    <nombre>Juan</nombre>
    <edad>30</edad>
    <link>/alumnos/Juan</link>
</alumno>
```

Una lista de alumnos tendrá el siguiente aspecto:

```
<alumnos>
  <alumno>
    <nombre>Juan</nombre>
    <edad>30</edad>
    <link>/alumnos/Juan</link>
  </alumno>
  <alumno>
    <nombre>Pedro</nombre>
    <edad>41</edad>
```

```
    <link>/alumnos/Pedro</link>
  </alumno>
</alumnos>
```

En JSON:

```
{
      "alumnos": {
            "alumno": [
                  {
                        "nombre": "Juan",
                        "edad": "30",
                        "link": "/alumnos/Juan"
                  },
                  {
                        "nombre": "Pedro",
                        "edad": "41",
                        "link": "/alumnos/Pedro"
                  }
            ]
      }
}
```

Una vez definida la representación, las URIs tienen la forma:

- *http://restfultai.com/alumnos*: para acceder a la lista de alumnos.
- *http://restfultai.com/alumnos/{nombre}*: para acceder a un alumno específico con el identificador con el valor *nombre*.

Ahora, se lanzarán peticiones sobre nuestro servicio. Por ejemplo, si queremos recuperar la información de un alumno con el nombre *Juan*, realizamos una petición a la URI *http://restfultai.com/alumnos/Juan*.

Una representación de *Juan* en el momento de la petición puede ser ésta:

```
<alumno>
    <nombre>Juan</nombre>
    <edad>30</edad>
    <link>/alumnos/Juan</link>
</alumno>
```

También se puede acceder a una lista de estudiantes a través de la URI *http://restfultai.com/alumnos* y la respuesta del servicio sería algo similar a ésta (asumiendo que solamente hay dos alumnos):

```
<alumnos>
  <alumno>
    <nombre>Juan</nombre>
    <edad>30</edad>
    <link>/alumnos/Juan</link>
  </alumno>
  <alumno>
    <nombre>Pedro</nombre>
    <edad>41</edad>
    <link>/alumnos/Pedro</link>
```

```
  </alumno>
</alumnos>
```

Funcionamiento

1. Un cliente realiza una petición HTTP con el método GET y *Juan* es el identificador del alumno.
2. El cliente establece la representación solicitada a través del campo de cabecera Accept (/alumnos/Juan).
3. El servidor web recibe e interpreta la petición GET como una acción RETRIEVE. En este momento, el servidor web cede el control al framework RESTful para gestionar la petición. Subrayar que los frameworks RESTful no recuperan de forma automática los recursos, ése no es su trabajo. La función del framework es facilitar la implementación de las restricciones REST. La lógica de negocio y la implementación del almacenamiento es el papel del código específico del dominio.
4. El servidor busca el recurso *Juan*. Encontrar el recurso podría significar buscarlo en una base de datos, un sistema de ficheros, o una llamada a otro servicio web.
5. Una vez que el programa encuentra a *Juan*, convierte el dato binario del recurso a la representación solicitada por el cliente.
6. Con la representación convertida a XML, el servidor envía de vuelta una respuesta HTTP con un código numérico de 200 (Ok) junto con la representación solicitada. Si hay algún error, el servidor HTTP devuelve el código numérico correspondiente, pero es el cliente el que debe tratar de forma adecuada el fallo. El fallo más común es que el recurso no exista, en cuyo caso se devolvería el código 404 (Not Found).

Todos los mensajes entre el cliente y el servidor son llamadas del protocolo estándar HTTP. Para cada acción de recuperación, enviamos una petición GET y obtenemos una respuesta HTTP con la representación del recurso solicitada, o bien, si hay un fallo, el correspondiente código de error: 403 Acceso prohibido, 404 Not Found si un recurso no se encuentra o 500 Internal Server Error si hay un problema con el código en forma de una excepción, entre otros.

En las peticiones de recuperación de datos resulta recomendable también implementar un sistema de caché. Para hacer esto utilizaremos el código de respuesta 304 Not Modified en caso de que los datos no hubiesen cambiado desde la última petición que realizamos (se podría pasar un parámetro con la fecha en la que obtuvimos la representación por última vez). De esta forma, si un cliente recibe ese código como respuesta, sabe que puede seguir trabajando con la representación de la que ya dispone, sin tener que descargar una nueva. Solicitar una representación para todos los alumnos funciona de forma similar.

El método HTTP GET solamente debería utilizarse para recuperar representaciones. Podríamos utilizar una petición GET para actualizar el estado de los datos en el servidor, pero no es recomendable. Una operación GET debe ser **SEGURA** e **IDEMPOTENTE**.

Para que una petición sea **segura**, múltiples peticiones al mismo recurso no deben cambiar el estado de los datos en el servidor. Por ejemplo, supongamos una petición en el instante t1

para un recurso R devuelve R1; a continuación, una petición en el instante t2 para R devuelve R2; suponiendo que no hay más acciones de modificación entre t1 y t2, entonces R1 = R2 = R.

Para que una petición sea **idempotente** tiene que ocurrir que múltiples llamadas a la misma acción dejan siempre el mismo estado en el recurso. Por ejemplo, múltiples llamadas para crear un recurso R en los instantes t1, t2, y t3, darían como resultado que el recurso R existe sólo como R, y que las llamadas en los instantes t2 y t3 son ignoradas.

(2) POST

El método POST se utiliza para **CREAR** recursos. Utilizamos el método HTTP POST para crear un nuevo alumno. De nuevo, la URI para añadir un nuevo alumno a la lista será *http://restfultai.com/alumnos*. El tipo de método para la petición lo determina el cliente.

Se asume que el alumno con nombre *Ana* no existe en la lista y se quiere añadirlo. La nueva representación XML de *Ana* es:

```
<alumno>
   <nombre>Ana</nombre>
   <edad>33</edad>
   <link></link>
</alumno>
```

El elemento *link* forma parte de la representación, pero está vacío debido a que este valor se genera en tiempo de ejecución y no es creado por el cliente cuando envía la petición POST. Esto es solamente una convención para el ejemplo. Sin embargo, los clientes que utilizan el servicio web pueden especificar la estructura de las URIs.

Funcionamiento:

1. Un cliente realiza una petición HTTP a la URI *http://restfultai.com/alumnos*, con el método HTTP POST.
2. La petición POST incluye una representación en forma de XML de *Ana.*
3. El servidor web recibe la petición y delega en el framework REST para que la gestione. El código dentro del framework ejecuta los comandos adecuados para almacenar dicha representación (de nuevo, el dispositivo de almacenamiento puede ser cualquiera).
4. Una vez que se ha completado el almacenamiento del nuevo recurso, se envía una respuesta de vuelta: si no ha habido ningún error, se enviará el código 201 (Created). Si se produce un fallo, se enviará el código de error adecuado: 400 petición incorrecta, 403 Acceso prohibido, 500 Internal Server Error. Además, es posible devolver en la cabecera HTTP Location la URL que dará acceso al recurso recién creado.

   Location: http://restfultai.com/alumnos/Ana

Las peticiones POST **NO son IDEMPOTENTES**, por lo que, al invocar una misma llamada varias veces sobre un mismo recurso, el estado del recurso puede verse alterado en cada una de ellas. Por ejemplo, al ejecutar varias veces la acción POST con los datos del ejemplo anterior, se podría estar creando cada vez un nuevo alumno de nombre *Ana*, teniendo así varios

alumnos con el mismo nombre y edad (pero asociados a IDs distintos, por ejemplo: /Ana, /Ana2, /Ana3, etc).

(3) PUT

El método PUT se utiliza para **ACTUALIZAR** (modificar) recursos, o para crearlos si el recurso en la URI especificada no existiese previamente. Es decir, PUT se utiliza para establecer un determinado recurso, dada su URI, a la representación que proporcionemos, independientemente de que existiese o no. Para actualizar un recurso, primero necesitamos su representación en el cliente. En segundo lugar, en el lado del cliente actualizaremos el recurso con los nuevos valores deseados. Por último, actualizaremos el recurso mediante una petición PUT, adjuntando la representación correspondiente.

Para nuestro ejemplo, se omite la petición GET para recuperar a *Juan* del servicio web, ya que se ha visto con anterior. Se supone que se desea modificar la edad, y cambiarla de 30 a 29. La nueva representación será:

```
<alumno>
   <nombre>Juan</nombre>
   <edad>29</edad>
   <link>/alumnos/Juan</link>
</alumno>
```

Funcionamiento:

1. Un cliente realiza una petición HTTP PUT a la URI *http://restfultai.com/alumnos/Juan*, incluyendo la nueva definición XML.
2. El servidor web recibe la petición y delega en el framework REST para que la gestione. El código dentro del framework ejecuta los comandos adecuados para actualizar la representación de *Juan*.
3. Una vez que se ha completado la actualización, se envía una respuesta al cliente. Si el recurso que hemos enviado no existía previamente, se devolverá el código 201 (Created). En caso de que ya existiese, se podría devolver 200 (Ok) con el recurso actualizado como contenido, o simplemente 204 (No Content) para indicar que la operación se ha realizado correctamente sin devolver ningún contenido.

Muchas veces se confunden los métodos PUT y POST. El significado de estos métodos es el siguiente:

- **POST**: publica datos en un determinado recurso. El recurso debe existir previamente, y los datos enviados son añadidos a él. Por ejemplo, para añadir nuevos alumnos con POST es necesario hacerlo con el recurso lista de alumnos (/alumnos), ya que la URI del nuevo alumno todavía no existe. La operación **NO es idempotente**, es decir, si añadimos varias veces el mismo alumno aparecerá repetido en nuestra lista de alumnos con URIs distintas.

- **PUT**: hace que el recurso indicado tome como contenido los datos enviados. El recurso podría no existir previamente, y en caso de que existiese sería sobrescrito con la nueva información. A diferencia de POST, PUT **es IDEMPOTENTE**. Múltiples llamadas idénticas a la misma acción PUT siempre dejarán el recurso en el mismo estado. La acción se realiza sobre la URI concreta que queremos establecer (/alumnos/Juan), de forma que varias

llamadas consecutivas con los mismos datos tendrán el mismo efecto que realizar sólo una de ellas.

Se pueden añadir nuevos alumnos de dos formas diferentes. La primera de ellas es haciendo POST sobre el recurso que contiene la lista de alumnos:

```
POST /alumnos HTTP/2
```

También es posible mediante PUT sobre el recurso de un alumno concreto:

```
PUT /alumnos/Juan HTTP/2
```

Si *Juan* existía previamente, sobrescribirá sus datos, en otro caso, creará el nuevo recurso.

Al utilizar POST sobre un recurso concreto, si el recurso existiese podríamos realizar alguna operación que modifique sus datos, pero si no existiese nos daría un error, ya que no podemos hacer POST sobre un recurso inexistente.

```
POST /alumnos/Juan HTTP/2
```

El caso anterior sólo será correcto si *Juan* existe, en caso contrario obtendremos un error. Para crear nuevos recursos con POST debemos recurrir al recurso del conjunto de alumnos. Una diferencia entre estas dos formas alternativas de crear nuevos recursos es que con PUT podemos indicar explícitamente el identificador del recurso creado, mientras que con POST será el servidor quien lo decida.

(4) DELETE

El método DELETE se utiliza para **BORRAR** recursos.

Funcionamiento:

1. Un cliente realiza una petición DELETE a la URI *http://restfultai.com/alumnos/Juan*
2. El servidor web recibe la petición y delega en el framework REST para que la gestione. El código dentro del framework ejecuta los comandos adecuados para borrar la representación de *Juan*.
3. Una vez que se ha completado la actualización, se envía una respuesta al cliente. Se podría devolver 200 (Ok) con el recurso borrado como contenido, o simplemente 204 (No Content) para indicar que la operación se ha realizado correctamente sin devolver ningún contenido.

Hasta aquí hemos visto las principales acciones que podemos realizar con los recursos en un servicio web RESTful. No conocemos cómo el servicio web implementa el almacenamiento de los datos, y no conocemos qué tecnologías se utilizan para implementar el servicio. Todo lo que conocemos es que nuestro cliente y servidor se comunican a través de HTTP, que usamos dicho protocolo de comunicaciones para enviar peticiones, y que nuestras representaciones de los recursos se intercambian entre el cliente y el servidor a través del intercambio de URIs.

2. OAuth 2.0

OAuth 2.0 (Open Authorization) es un protocolo de AUTORIZACIÓN y NO es un protocolo de autenticación.

Este estándar abierto y está diseñado principalmente como un medio para conceder acceso a un conjunto de recursos (por ejemplo, API remotas o datos de usuario), es decir, para permitir que un sitio web o una aplicación accedan a recursos alojados por otras aplicaciones web en nombre de un usuario. Proporciona acceso consentido y restringe las acciones que la aplicación del cliente puede realizar en los recursos en nombre del usuario, sin compartir nunca las credenciales del usuario.

Está basado en **tokens de acceso**, esto es, un dato que representa la autorización para acceder a los recursos en nombre del usuario final. OAuth 2.0 no define un formato específico para los tokens de acceso. Sin embargo, en algunos contextos, se suele utilizar el formato JSON Web Token (JWT).

La solicitud, el intercambio y la respuesta de los tokens siguen el siguiente flujo general:

1. El cliente solicita autorización (SOLICITUD DE AUTORIZACIÓN) al servidor de autorización, proporcionando el id y el client secret como identificación; también proporciona los ámbitos y un URI de extremo (URI de redireccionamiento) al que enviar el token de acceso o el código de autorización.
2. El servidor de autorización autentica al cliente y verifica que los ámbitos solicitados están permitidos.
3. El propietario del recurso interactúa con el servidor de autorización para conceder el acceso.
4. El servidor de autorización redirige de vuelta al cliente un código de autorización o un TOKEN DE ACCESO, según el tipo de concesión.

3. JWT

JSON Web Token (**JWT**) es un estándar abierto (RFC 7519) que define una forma compacta (la información se transmite a través de una URL o dentro de una cabecera HTTP) y autónoma (la carga útil tiene toda la información del usuario, no siendo necesario consultar una BD) de transmitir información de forma segura entre las partes como un objeto JSON. Esta información se puede verificar y confiar porque está firmada digitalmente. Los JWT se pueden firmar usando un secreto (con el algoritmo HMAC) o un par de claves pública/privada usando RSA.

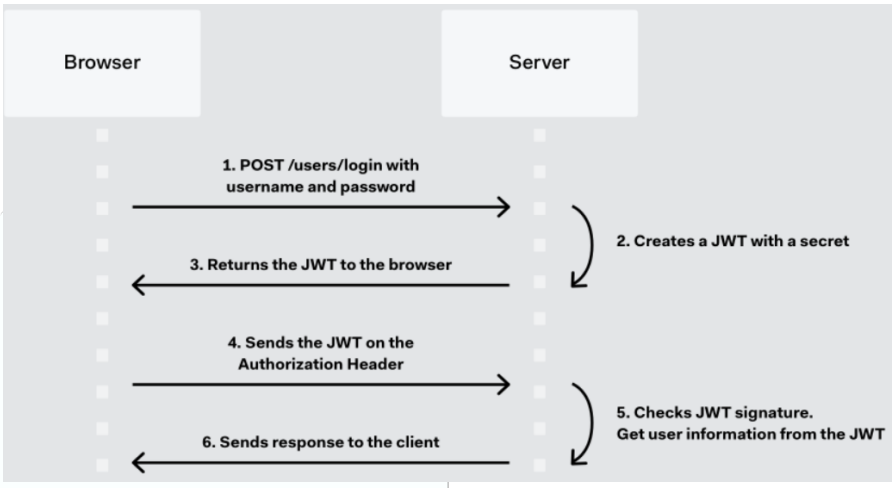
En la autenticación con token, el usuario se identifica bien con un usuario/contraseña o mediante una única clave y la aplicación web le devuelve una especie de firma cifrada que el usuario usará en las cabeceras de cada una de las peticiones HTTP.

JWT es un mecanismo de **AUTENTICACIÓN sin estado** ya que el estado del usuario nunca se almacena en el servidor, sino que se guarda en el cliente y es la aplicación la que comprobará

si es válida en cada una de las peticiones. Este mecanismo es opuesto a la autenticación a través de sesión, en el que sí se almacenan los datos en el servidor.

En la autenticación, cuando el usuario inicia sesión correctamente con sus credenciales, se devolverá un JWT. Siempre que el usuario desee acceder a una ruta protegida, deberá enviar el JWT, normalmente en el encabezado **de autorización** utilizando el esquema **Bearer**. Por lo tanto, el contenido del encabezado será:

```
Authorization: Bearer <token>
```



Un JWT consta de tres partes, todas ellas codificadas en Base64 y separadas por un punto:

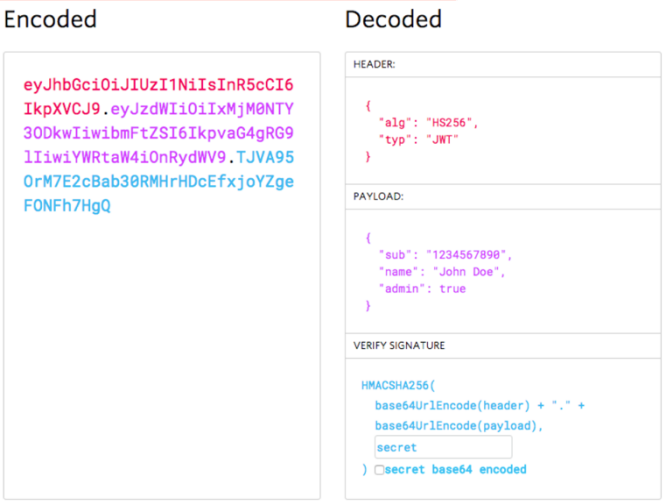
HEADER.PAYLOAD.SIGNATURE

- **HEADER (cabecera)**: tipo de token, que es JWT, y el algoritmo de firma que se utiliza, como HMAC SHA256 o RSA. La cabecera está codificada en Base64Url.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **PAYLOAD (carga útil)**: contiene los claims, que son declaraciones sobre una entidad (normalmente, el usuario) y datos adicionales. La carga útil se codifica en Base64Url.

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```



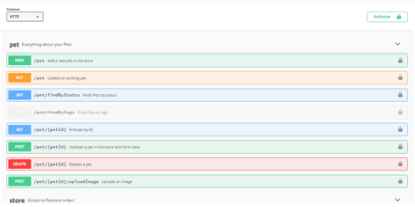
- **SIGNATURE (firma)**: para crear la firma se necesitan el Header, el Payload, un secreto y el algoritmo especificado en el encabezado. La firma se usa para verificar que el mensaje no se modificó en el camino y, en el caso de tokens firmados con una clave privada, también puede verificar que el remitente del JWT es quien dice ser.

4. Especificación de APIs: OpenAPI



La especificación **OpenAPI versión** 3 (**OAS3**), anteriormente Swagger, es un estándar que proporciona un medio coherente para transmitir información a través de cada etapa del ciclo de vida de la API. Es un lenguaje de especificación para las API HTTP que define la estructura y la sintaxis de una manera que no está ligada al lenguaje de programación en el que se crea la API. Las especificaciones de la API suelen estar escritas en YAML o JSON, lo que permite compartir y consumir la especificación con facilidad.

Con OpenAPI se puede descubrir rápidamente cómo funciona una API. Como no depende del lenguaje de programación, es fácil identificar y comprender rápidamente las capacidades del servicio. También se puede usar para configurar la infraestructura, generar código de cliente y crear casos de prueba para las API.



Swagger UI genera automáticamente a partir de la especificación OpenAPI (en YAML o JSON) la documentación visual de la API.

Otras herramientas para especificar APIs son:

- **RAML**: utiliza YAML para documentar.
- **API Blueprint**: la especificación se realiza en un documento de texto plano en formato Markdown (texto enriquecido).
- **WADL**: equivalente de WSDL para SOAP.

5. Ejemplos servicios web REST

Podemos citar como ejemplos:

- X: https://developer.twitter.com/en/docs/api-reference-index
- GMAIL: https://developers.google.com/gmail/api/v1/reference
- BOOKING: https://connect.booking.com/user_guide/site/en-US/
- NETFLIX: https://partnerhelp.netflixstudios.com/hc/en-us/articles/225591608-Backlot-Source-Requests-Delivery-Management-API-
- AEMET: https://opendata.aemet.es/dist/index.html
- API REST pública para pruebas: https://jsonplaceholder.typicode.com (probar con extensión Advanced REST Client de Chrome).

6. Códigos de estado HTTP

1×× Informational

100 Continue
101 Switching Protocols
102 Processing

2×× Success

200 OK
201 Created
202 Accepted
203 Non-authoritative Information
204 No Content
205 Reset Content
206 Partial Content
207 Multi-Status
208 Already Reported
226 IM Used

3×× Redirection

300 Multiple Choices
301 Moved Permanently
302 Found
303 See Other
304 Not Modified
305 Use Proxy
307 Temporary Redirect
308 Permanent Redirect

4×× Client Error

400 Bad Request
401 Unauthorized
402 Payment Required
403 Forbidden
404 Not Found
405 Method Not Allowed
406 Not Acceptable
407 Proxy Authentication Required
408 Request Timeout
409 Conflict
410 Gone
411 Length Required
412 Precondition Failed
413 Payload Too Large
414 Request-URI Too Long
415 Unsupported Media Type
416 Requested Range Not Satisfiable
417 Expectation Failed
418 I'm a teapot
421 Misdirected Request
422 Unprocessable Entity
423 Locked
424 Failed Dependency
426 Upgrade Required
428 Precondition Required
429 Too Many Requests
431 Request Header Fields Too Large
444 Connection Closed Without Response
451 Unavailable For Legal Reasons
499 Client Closed Request

5×× Server Error

500 Internal Server Error
501 Not Implemented
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
505 HTTP Version Not Supported
506 Variant Also Negotiates
507 Insufficient Storage
508 Loop Detected
510 Not Extended
511 Network Authentication Required
599 Network Connect Timeout Error

7. Servicios web RESTful vs servicios web SOAP

A continuación, se muestra una tabla resumen con las principales diferencias entre los servicios RESTful y los servicios SOAP.

RESTful	SOAP
Ligeros	Pesados
Se trata de un estilo arquitectónico	Se definen de forma estándar
No necesitan herramientas adicionales para construirlos	Contamos con herramientas que facilitan el trabajo
Protocolo de comunicaciones entendible por un humano	Se pueden integrar de forma automática en aplicaciones
El desarrollador del servicio debe dejar documentación, ejemplos de uso y librerías para acceder al servicio	Interoperables

A la hora de decidir si implementar nuestros servicios web de tipo SOAP o RESTful, podemos tener en cuenta los siguientes factores:

RESTful	SOAP
Si tenemos servicios sin estado	Si necesitamos un contrato formal entre cliente y servicio (WSDL)
Si devuelve datos no dinámicos, que puedan ser almacenados en la caché del navegador	Si queremos interconectar sistemas sin tener que definir nosotros el formato para el intercambio de datos
Si el ancho de banda es limitado (dispositivos móviles)	Si queremos que los servicios sean capaces de almacenar información de estado
Si queremos incluirlos en documentos web (por ejemplo, mediante JS)	Si vamos a utilizar métodos de comunicación asíncronos
Si existe un entendimiento entre productor y consumidor respecto a la forma de intercambiar los datos, o bien se quiere proporcionar un *toolkit* de desarrollo para los clientes	Si necesitamos otras características avanzadas como el envío fiable de mensajes, manejo de transacciones, seguridad, etc.
En arquitecturas orientadas a microservicios	Integración con servicios de la AGE

7. FRAMEWORKS

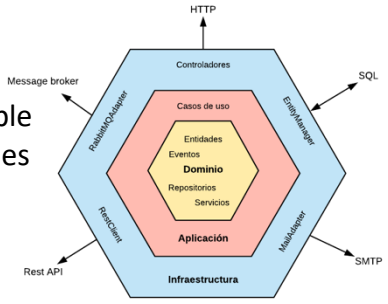
Destacamos los siguientes frameworks:

- **Apache Axis2**: framework para desarrollo de servicios web SOAP y RESTful con implementaciones para Java y C.
- **Apache CXF**: framework para Java que soporta servicios web SOAP y RESTful.
- **Jersey**: framework de Eclipse con soporte para servicios web RESTful en Java.
- **RESTEasy**: framework para desarrollo de servicios web RESTful en Java (JAX-RS).
- **WCF**: parte de .NET para desarrollo de servicios web SOAP y RESTful.
- **gSOAP**: framework para servicios web SOAP en C/C++.
- **CodeIgniter**: framework para PHP que soporta servicios web SOAP.
- **Zend**: framework para PHP que soporta servicios web SOAP y RESTful.
- **Laravel**: framework para PHP que soporta servicios web SOAP y RESTful.
- **SoapUI**: herramienta para probar servicios web SOAP y RESTful.
- **Postman**: herramienta para probar servicios web RESTful.
- **OpenAPI (antes Swagger)**: especificación para desarrollo de servicios web RESTful.

8. ARQUITECTURA HEXAGONAL

La arquitectura hexagonal, también conocida como arquitectura de puertos y adaptadores, tiene como principal motivación separar una aplicación software en distintas capas, cada una con su propia responsabilidad. De esta forma consigue desacoplar las capas permitiendo que evolucionen de manera separada y fomentando la reutilización.

Capas de la arquitectura hexagonal (igual que en DDD):



- **Dominio**: es el núcleo de la aplicación conteniendo las reglas de negocio que definen el comportamiento y las restricciones. Además, es responsable de mantener la consistencia del estado de los objetos del dominio (clases modelos).
- **Aplicación**: son los servicios que definen los puntos de entrada a la aplicación software.
- **Infraestructura**: permite la comunicación con componentes externos (BDs, WS, IUs, etc.).

Los puertos y adaptadores pueden ser definidos como sigue:

- **Puerto**: definición de una interfaz pública (como una interface en Java).
- **Adaptador**: especialización de un puerto para un contexto concreto, también denominado controlador (como una clase que implementa una interface).

La arquitectura hexagonal propone que el dominio sea el núcleo de las capas y que no se acople a nada externo. Gracias al principio de inversión de dependencias (principio perteneciente a SOLID) el acoplamiento se produce con contratos (interfaces o puertos) y no con implementaciones concretas.

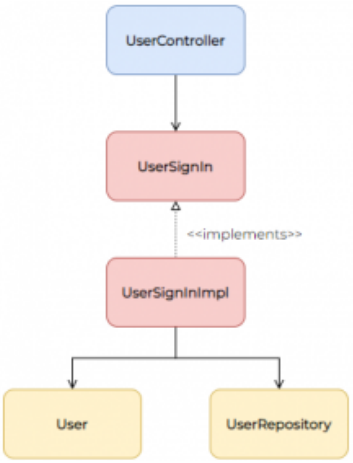
Lo que se persigue es que el núcleo (dominio) sea visto como una API con unos contratos bien especificados. Definiendo puertos o puntos de entrada e interfaces (adaptadores) para que otros módulos (UI, BD, test, WS…) puedan implementarlas y comunicarse con la capa de negocio sin que ésta deba saber el origen de la conexión.

Aunque se suele representar como forma de hexágono, realmente no se limita el número de interacciones entre las capas. De esta manera, cada lado del polígono sería un puerto al que nos podríamos conectar a través de un adaptador.

Cuando cualquier controlador quiere usar la aplicación en un puerto, envía una solicitud que es convertida por un adaptador para la tecnología específica del controlador en una llamada o mensaje de procedimiento utilizable, que lo pasa al puerto de la aplicación. La aplicación es ignorante de la tecnología del adaptador. Cuando la aplicación tiene algo que enviar, lo envía a través de un puerto a un adaptador, lo que crea las señales apropiadas que necesita la tecnología receptora (humana o automatizada). La aplicación tiene una interacción semánticamente sólida con los adaptadores en todos sus lados, sin conocer realmente la naturaleza de las cosas en el otro lado de los adaptadores.

Ejemplo:

Implementamos un adaptador (clase UserController) en la capa de infraestructura para conectarnos al puerto HTTP del sistema.



Este adaptador necesitará utilizar un "caso de uso" de la capa de aplicación (interfaz UserSignIn y clase UserSignInImpl) para registrar un usuario (se define una interfaz y una clase que la implementa siguiendo el principio de inversión de dependencias). En este caso, el puerto sería la interfaz, mientras que el adaptador sería la implementación concreta.

Este caso de uso de la capa de aplicación necesitará crear una entidad usuario (clase User) y almacenarla en un repositorio (interfaz UserRepository), perteneciendo User y UserRepository a la capa de dominio.